



The Mobile Heart Box

Personal monitoring solution on the move

Fernando de Almeida Ferreira

Gabriel Lameira Marta

Jorge Filipe Saraiva Marques

Martim Peralta Gomes

Tiago Alexandre Vitória Salgueiro

Licenciatura em Engenharia de Computadores e Informática

Orientador: José Maria Amaral Fernandes

Uso de ferramentas de inteligência artificial

Este projeto fez uso de ferramentas de inteligência artificial, de seguida listadas com os devidos propósitos:

- **ChatGPT:** utilizado para propósitos de pesquisa e para um melhor entendimento sobre alguns componentes do projeto.
- **Gemini:** utilizado para o desenvolvimento de código.
- **Claude:** utilizado para desenvolvimento de código e para obter esclarecimentos sobre o funcionamento de alguns *frameworks* utilizados.
- **Grok:** utilizado como apoio à realização do relatório, oferecendo *guidelines* sobre a estrutura e realçando temas que necessitavam de ser explorados.

Importante realçar que a utilização destas ferramentas foi exatamente essa, utilizá-las como ferramentas e não como substitutas do nosso papel na realização deste projeto.

Resumo

Atualmente, vivemos numa sociedade em que cada vez mais as pessoas trabalham em frente a um computador e sentadas numa secretária. Essa realidade fez com que o número de pessoas preocupadas com o seu bem-estar tenha aumentado, levando mais pessoas a praticar desporto, e além disso, a quererem retirar informação dessa prática, para se poderem exercitar de uma forma mais consciente e adaptada. No nosso projeto, focamo-nos nos praticantes de *biking* e *hiking*. O facto de praticarem o seu desporto ao ar livre, com uma enorme diversidade de locais onde o podem fazer, torna especialmente importante a recolha de dados não só do praticante, mas também do ambiente que o rodeia.

Ao mesmo tempo que se verificou este fenómeno, os sensores fisiológicos e os dispositivos *wearables* foram sendo, cada vez mais, uma presença normal no quotidiano das pessoas. O aumento da autonomia e da capacidade de transferir dados nesses dispositivos surgem como principal motivo para o aumento dessa presença. No entanto, a utilização desses dispositivos como soluções individuais tem pouco ou nenhum interesse. É na sua combinação com outros dispositivos, principalmente *smartphones*, que os sensores fisiológicos e *wearables* podem atingir o seu potencial máximo. A capacidade de processamento dos *smartphones*, que está em constante crescimento, aliada aos seus diversos recursos de conectividade, permitem criar uma grande variedade de soluções.

Este projeto pretende desenvolver uma solução móvel que permita a praticantes *hiking* e *biking* monitorizarem dados fisiológicos e ambientais durante a prática desportiva, assim como analisarem dados sobre a mesma, antes (análise de atividades anteriores) e depois de a realizarem. Pretendemos oferecer um protótipo que responda às necessidades do nosso público-alvo e que combine a utilidade dos sensores fisiológicos, dos dispositivos *wearables* e dos *smartphones* de uma forma inovadora e eficaz.

Agradecimentos

Gostaríamos de agradecer ao nosso professor orientador, José Maria Amaral Fernandes, pela disponibilidade e ajuda prestada. Aos nossos professores da cadeira de PECEI, José Manuel Matos Moreira e Rui Luís Andrade Aguiar, agradecemos pelas linhas orientadoras e ideias que contribuíram para o desenvolvimento deste projeto. Estendemos os nossos agradecimentos a todos os docentes que, durante o nosso percurso na licenciatura em Engenharia de Computadores e Informática, contribuíram para o nosso desenvolvimento académico e pessoal.

A todos os nossos colegas que nos ajudaram, trocando opiniões e conhecimentos, um obrigado pelo contributo importante que tiveram no desenvolvimento deste projeto.

Por fim, a todos os familiares e amigos que de alguma forma contribuíram para este projeto, um agradecimento especial.

A equipa do Mobile Heart Box

Conteúdo

1	Introdução	4
1.1	Contexto	6
1.2	Motivação	6
1.3	Objetivos.....	7
2	State of the Art.....	8
2.1	Projetos relacionados.....	8
2.1.1	The Mobile Heart Box 24/25.....	8
2.1.2	Hexoskin Smart Shirts.....	9
2.1.3	Smarwatches.....	9
2.1.4	Sistemas de navegação de automóveis	10
2.2	Tecnologia	11
2.2.1	ESP32-CAM.....	11
2.2.2	Sensor PPG	12
2.2.3	ESP32-S3.....	13
2.2.4	ESP32-S3-Zero	13
2.2.5	Sensor MLX90640	14
2.2.6	RTC DS3231	15
2.2.7	Sensor IMU	16
2.2.8	Sensor mmWave HMMD	16
2.2.9	Sensor 60GHz mmWave Radar.....	17
2.2.10	Sensor AD8232	18
2.2.11	Bluetooth Low Energy	19
2.2.12	Broker MQTT Eclipse Mosquitto.....	19
3	Requisitos do sistema e arquitetura	20

3.1	Requisitos do sistema	20
3.1.1	Recolha de requisitos.....	20
3.1.2	Atores	20
3.1.3	Casos de Uso	21
3.1.4	Requisitos Funcionais.....	21
3.1.5	Requisitos Não Funcionais	22
3.1.6	Pressupostos e Dependências	23
3.2	Arquitetura do Sistema	24
4	Implementação.....	27
4.1	Visão geral.....	27
4.2	Hardware	28
4.2.1	Caixa dianteira.....	28
4.2.2	Caixa traseira	30
4.2.3	Integração de sensores	32
4.2.4	Esquemas elétricos	33
4.2.5	Alimentação e gestão energética	37
4.3	Firmware.....	38
4.3.1	ESP32-CAM.....	38
4.3.2	ESP32-S3.....	39
4.3.3	ESP32-S3-Zero.....	41
4.4	Aplicação móvel.....	43
4.4.1	Flutter	43
4.4.2	Arquitetura da aplicação	44
4.4.3	Gestão de estado	46
4.4.4	Comunicação com o broker MQTT.....	47
4.4.5	Ecrã de monitorização da atividade.....	49
4.4.6	Configuração via QR Code	52

4.4.7Histórico de Atividades	54
4.5Backend	57
4.5.1Receção de Dados em Tempo Real (Mosquitto)	57
4.5.2Middleware do Sistema (Node-RED).....	58
4.5.3Persistência de Séries Temporais (InfluxDB)	59
4.5.4Monitorização (Grafana)	60
4.6Desafios e Soluções	61
5Testes e Resultados.....	62
5.1 Testes Funcionais	62
5.2 Testes de Integração e Comunicação.....	64
5.3 Testes Não Funcionais.....	66
6Conclusões.....	67
6.1Trabalho Futuro	68
Referências	69

1 Introdução

Num estudo realizado na década passada por investigadores norte-americanos, onde se procurava encontrar motivos para o aumento da obesidade na população americana, verificou-se que, durante as 5 décadas anteriores, existiu uma redução de cerca de 30% no número de trabalhadores que tinham empregos fisicamente exigentes. Outro estudo, desta vez europeu, concluiu que é nos países em que a digitalização estava mais avançada, que as pessoas passam mais tempo sentadas (principalmente nos seus empregos). A tabela seguinte pretende ilustrar estas conclusões em números concretos, para facilitar a visualização (Tab 1.1 e 1.2):

Tabela 1.1: Evolução da % de trabalhos sedentários [1]

<i>Ano</i>	<i>Trabalhos ativos</i>	<i>Trabalhos sedentários</i>
1960	48%	52%
2008	20%	80%

Tabela 1.2: Evolução na % de adultos que passam >4,5h/dia sentados [2]

<i>Ano</i>	<i>Adultos que passam >4,5/dia sentados</i>
2002	49,3%
2005	50,2%
2013	52,0%
2017	54,3%

Em paralelo com o aumento do sedentarismo, principalmente nas horas ocupadas pelo trabalho, verificou-se um aumento na prática desportiva recreativa. Um estudo realizado em Espanha e publicado pela BMC Public Health, analisou dados relativos à população espanhola entre 1987 e 2006, verificando que, enquanto o sedentarismo aumentava durante as horas de trabalho, as atividades de lazer sedentárias diminuía, como mostra a tabela 1.3:

Tabela 1.3: Evolução do número de atividades sedentárias no lazer [3]

Ano	% de atividades sedentárias no lazer	
	Homens	Mulheres
1987	50%	67%
2006	45%	63%

Este fenómeno foi também verificado em diversos estudos e relatórios da OCDE [4], que verificaram que os níveis de atividade física têm vindo a diminuir no trabalho e a aumentar nos tempos de lazer, e também que, entre 2017 e 2022, houve um aumento de 9% no número de adultos que praticam desporto de forma regular.

Outros estudos focaram-se em mostrar a relação entre o aumento do sedentarismo em horário de trabalho e o aumento da prática desportiva em tempos de lazer. Um estudo publicado pelo American Journal of Preventive Medicine, demonstrou que os trabalhadores com empregos mais sedentários têm uma maior tendência a praticar desporto nas suas horas livres, funcionando como um mecanismo compensatório (Tab 1.4). Estes trabalhadores tornaram-se mais dependentes da prática desportiva nos seus tempos de lazer, para poderem manter níveis de atividade saudáveis.

Tabela 1.4: Probabilidade de praticar desporto nas horas de lazer [5]

Tipo de atividade no trabalho	% de trabalhadores com esse tipo de trabalho	Probabilidade de praticar desporto
Trabalho sentado	42-47%	Elevada
Trabalho em pé	27-30%	Moderada
Trabalho com caminhada	15-20%	Moderada
Trabalho pesado	5-10%	Baixa

Sendo então perceptível que a prática desportiva tem vindo a aumentar, é importante tentar encontrar que tipos de desportos as pessoas tendem a praticar, principalmente as que se enquadram neste público-alvo da classe trabalhadora. Após uma pesquisa, foi-nos permitido concluir que o *hiking* e o *biking* eram dois desses desportos. Num estudo realizado por duas agências do desporto *outdoor* americano [7], mais de 100 milhões de residentes nos EUA praticavam esses dois desportos (cerca de 35% da população), sendo *hiking* a atividade *outdoor* mais popular. Já um estudo europeu [8], determinou que cerca de 82% dos europeus participaram de alguma forma em atividades *outdoor* no último ano, com o *hiking* e o *biking* a representar mais de 60% dessa participação.

Portanto, surge a questão de como podemos melhorar essa mesma prática desportiva. Os recentes avanços tecnológicos na área dos sensores e da monitorização fisiológica ofereceram várias formas de poder acompanhar e recolher informação sobre a atividade física relacionada, de forma a controlar diversas variáveis, medir progresso e gerir cargas.

1.1 Contexto

Este projeto foi desenvolvido no âmbito do Projeto da Licenciatura em Engenharia de Computadores e Informática, por alunos do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro. O foco do projeto é aplicar conceitos adquiridos ao longo da licenciatura, com especial atenção à IoT, segurança informática e nas comunicações, redes de comunicações, programação, análise de sistemas e interação humano-computador.

1.2 Motivação

As motivações deste projeto são, através da aplicação dos conceitos adquiridos nas áreas acima referidas, as seguintes:

- Promover uma prática desportiva mais informada e consciente, através da monitorização e acompanhamento da mesma.
- Ao oferecer uma solução interativa e de fácil utilização, contribuir para o crescente interesse em praticar desporto e seguir um estilo de vida saudável.

- Ajudar a desenvolver o autoconhecimento e atingir objetivos, através do acompanhamento do progresso, de forma a poder oferecer aos praticantes de *hiking* e *biking* um sentido de propósito.
- Tirar proveito, e ao mesmo tempo contribuir, para a crescente inovação tecnológica na área dos sensores, monitorização, segurança e conectividade dos dispositivos.

Todas estas motivações têm o objetivo comum de melhorar a experiência desportiva dos praticantes de *hiking* e *biking*. Vemos a nossa solução como algo que possa acompanhar todos os praticantes, desde os que praticam por motivos de saúde ou lazer, até aos que têm objetivos e procuram atingir algum tipo de sucesso desportivo, durante a sua prática desportiva.

1.3 Objetivos

Neste projeto, pretendemos atingir dois tipos de objetivos: os académicos, referidos no capítulo 1.1; e os objetivos do projeto em si, que serão desenvolvidos de seguida.

Pretendemos então desenvolver uma solução móvel que nos permita recolher dados fisiológicos (como os batimentos cardíacos, padrões respiratórios, temperatura corporal, entre outros) e ambientais (distâncias, inclinação, localização, entre outros), durante a atividade física praticada no exterior. A nossa solução tem como objetivo ser utilizada por praticantes de *hiking* e *biking*, de forma a contribuir para uma melhor prática desportiva e uma maior consciência sobre o seu corpo e capacidades físicas.

A nível tecnológico, é pretendido que este sistema seja capaz de integrar múltiplos sensores em sistemas baseados nos microcontroladores ESP32. Isto fará com que consigamos atribuir utilidade aos dados obtidos pelos sensores, coordenando-os através dos ESP32, uma vez que, utilizados individualmente, os sensores não teriam grande relevância. O uso de microcontroladores torna-se também importante para que a transmissão de dados em tempo real possa ser atingida, com o objetivo desses dados poderem ser visualizados numa aplicação externa, presente num *smartphone*.

2 State of the Art

2.1 Projetos relacionados

Nesta secção, encontram-se alguns projetos que foram desenvolvidos no mesmo domínio que a Mobile Heart Box. Foram importantes para perceber em que aspetos seria possível evoluir e inovar, assim como para perceber que ideias e conceitos já funcionam e fazem sentido ser reaproveitadas.

2.1.1 The Mobile Heart Box 24/25

Este projeto [9], desenvolvido por alunos no mesmo contexto em que a nossa equipa está inserida, serviu como grande base do nosso trabalho. Foi desenvolvido durante o ano letivo anterior, e tinha como grande objetivo a monitorização fisiológica e ambiental para praticantes de *cicling* (bicicleta em ambientes fechados). O nosso projeto surgiu como uma migração para um contexto móvel e exterior, o que traz diversos desafios.

Esta migração permitiu-nos aproveitar várias partes do trabalho desenvolvido para o contexto do nosso projeto, mas com vários obstáculos que necessitam de ser ultrapassados. A adaptação dos sensores ao ambiente exterior e a integração de novos sensores de uma forma simples e flexível, de forma que possa ser utilizada em constante movimento, surgem como dois dos maiores problemas. A necessidade de conseguir atingir uma autonomia energética que seja ajustada ao contexto de uso da nossa solução, é outro dos desafios que teremos pela frente durante esta migração.

2.1.2 Hexoskin Smart Shirts

Desenvolvida pela empresa canadiana Hexoskin, as Hexoskin Smart Shirts [10] são um conjunto de peças de vestuário inteligente, que podem ser usadas durante o dia-a-dia e durante a atividade física. Tal como a nossa solução, integram uma aplicação externa para a visualização e análise de dados lidos pelos sensores, que estão embebidos no material têxtil do vestuário.

Focam-se na recolha de dados cardíacos e respiratórios, com o objetivo de analisar padrões de sono e de atividade física. Estas medidas são usadas para controlo de estado de saúde do utilizador, melhorias dos níveis de performance desportiva, investigação clínica e como ferramenta de monitorização dos membros de equipas de resposta rápida, como bombeiros ou membros do exército.

Este projeto foi útil para perceber como seria possível integrar sensores numa solução móvel, assim como otimizar a conectividade com outros dispositivos para poder visualizar e analisar dados. No entanto, não é capaz de obter informação sobre o ambiente em que o praticante se encontra, o que é fulcral no nosso projeto.

2.1.3 Smarwatches

Os *smartwatches* são cada vez mais, um dos dispositivos *wearable* mais presentes no quotidiano das pessoas. Muitos deles já são capazes de fazer leituras fisiológicas, assim como comunicá-las a outros dispositivos, como *smartphones*, através do Bluetooth. Esses dois aspetos foram importantes estudar e analisar, para perceber o que podia ser transferido para o nosso projeto.

No entanto, sendo dispositivos já bastante presentes no dia-a-dia das pessoas, surgem como competidores, pelo que também foi necessário tentar perceber o que podíamos fazer de diferente. Desde logo surge a questão dos custos. Muitos dos *smartwatches* capazes de efetuar medições fisiológicas têm preços elevados, tornando-os pouco disponíveis para grande parte da população. Além disso, são poucos ou nenhuns os *smartwatches* capazes de efetuar medições ambientais. Esses dois motivos surgem como a nossa vantagem competitiva em relação a este tipo de dispositivos.

2.1.4 Sistemas de navegação de automóveis

Foi interessante estudar este tipo de sistemas, pois apesar de muito diferentes da nossa solução, respondem a uma necessidade fulcral do nosso produto, a capacidade de efetuar medições ambientais. Esta característica dos sistemas de navegação não é encontrada em nenhum dos domínios tecnológicos anteriormente referidos, pelo que foi necessário afastarmo-nos ligeiramente da área tecnológica do nosso produto para podermos retirar algum conhecimento sobre este tipo de medições.

Durante a nossa pesquisa, tentamos perceber de que forma estes sistemas integrados em automóveis eram capazes de recolher dados relativos à temperatura ambiente, a forma como integram módulos de GPS e a utilidade dos sensores de proximidade para a deteção de situações de perigo. Além disto, o estudo destes sistemas foi útil para perceber como agilizar a recolha de dados através de sensores com uma comunicação eficiente com um dispositivo móvel, algo que também é importante no nosso projeto e que está presente em muitos automóveis.

2.2 Tecnologia

Neste capítulo abordaremos as principais tecnologias utilizadas no nosso projeto, assim como o porquê da escolha das mesmas.

2.2.1 ESP32-CAM

O ESP32-CAM (Fig 2.2.1.1) é um microcontrolador com WiFi e Bluetooth (BLE) integrados, com suporte para câmara, que limita o número de pinos disponíveis. Contém um *slot* para um cartão microSD, o que permite guardar imagens e vídeos, assim como dados provenientes de sensores.

A nível energético, o ESP32-CAM conta com modos de baixo consumo que permitem um maior controlo sobre a autonomia energética do sistema. Este microcontrolador conta com recursos de conectividade muito flexíveis, um dos principais motivos pelo qual o escolhemos usar no nosso projeto. É capaz de enviar dados para a *cloud* e de comunicar via MQTT/HTTP. É também possível realizar processamento local (desde que simples), permitindo a deteção de movimento.

Figura 2.2.1.1: Microcontrolador ESP32-CAM



2.2.2 Sensor PPG

Sensor de fotopletismografia – dispositivo ótico capaz de medir variações no volume de sangue nos tecidos, o que permite monitorizar parâmetros fisiológicos de forma não invasiva, daí a sua utilidade em mobilidade. O sensor PPG (Fig 2.2.2.1) conta com uma luz LED, que ilumina a pele. O sangue absorve parte da luz e a outra parte é refletida. Um fotodetetor mede a luz refletida, e as variações desses valores permitem calcular o pulso sanguíneo.

Através deste mecanismo, o sensor PPG é capaz de medir a frequência cardíaca e a variabilidade da mesma, assim como estimativas da taxa respiratória e da saturação do oxigénio. Estes sensores comunicam por I2C e apresentam um consumo energético muito reduzido e configurável. O sinal recolhido pelo sensor necessita de processamento para extrair informação útil: filtragem de ruído, deteção de picos e cálculo de métricas.

Este sensor foi apenas utilizado para testes, uma vez que tinha uma limitação que fazia com que perdesse a utilidade no contexto do nosso projeto: o sensor era muito sensível, o que fazia com que ao pressionar com um pouco menos de intensidade ele deixava de enviar valores.

Figura 2.2.2.1: Sensor PPG

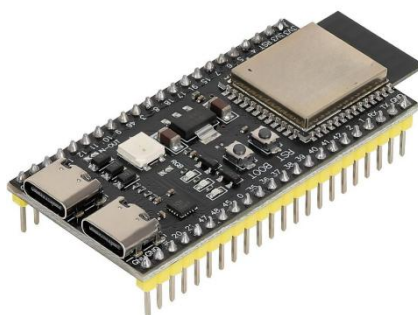


2.2.3 ESP32-S3

Tal como o ESP32-CAM, o ESP32-S3 (Fig 2.2.3.1) é um microcontrolador desenvolvido pela Espressif Systems para aplicações de IoT, inteligência artificial e processamento local. Destaca-se pela sua maior capacidade de processamento e por uma melhor gestão dos periféricos, ponto fulcral para uma integração com sensores mais eficaz. Tem conectividade com dispositivos externos através do WiFi e do BLE.

O ESP32-S3 está otimizado de forma a poder atingir um baixo consumo energético, através de modos de consumo como o *light sleep* e o *deep sleep*. Tem suporte para memórias *flash* externas. Além disso a sua conectividade WiFi, permite-o comunicar através de MQTT/HTTP, tanto como servidor como cliente, e oferece também a possibilidade de integração com *clouds*.

Figura 2.2.3.1: Microcontrolador ESP32-S3



2.2.4 ESP32-S3-Zero

O ESP32-S3-Zero (Fig 2.2.4.1) é um módulo mais compacto baseado no microcontrolador ESP32-S3 acima referido. Este módulo contém um processador dual-core Xtensa LX7 de 32 bits, capaz de atingir frequências de 240 MHz, o que lhe oferece um excelente desempenho em aplicações de processamento de dados.

A nível de conectividade, o S3-Zero suporta a comunicação através de WiFi e BLE, enquadrando-se assim com os restantes componentes do projeto. Integra também um leque de interfaces bastante flexível para a comunicação com periféricos.

Em termos de consumo energético, inclui vários modos de consumo reduzido, o que o torna indicado para dispositivos alimentados a bateria.

Figura 2.2.4.1: Módulo ESP32-S3-Zero



2.2.5 Sensor MLX90640

O sensor MLX90640 (Fig 2.2.5.1) é um sensor que integra uma câmara térmica de infravermelhos, capaz de medir a temperatura à distância, sem contacto direto. É utilizado em diversos contextos, como na monitorização térmica, na deteção de presença e em várias aplicações industriais e sistemas IoT. Consegue efetuar medições entre os 40°C negativos e os 300°C positivos. A sua integração com microcontroladores é possível através de uma interface de comunicação no protocolo I2C.

O sensor capta a radiação térmica emitida pelos corpos, converte-a em valores de temperatura e gera uma imagem térmica. O seu consumo energético varia com a taxa de atualização, mas é no geral, bastante reduzido, o que torna este sensor ideal para sistemas IoT e para o nosso projeto, que necessita de ser móvel e em que a autonomia energética é crucial.

Figura 2.2.5.1: Sensor MLX90640



2.2.6 RTC DS3231

O DS3231 (Figura 2.2.6.1) é um módulo de relógio de tempo real de elevada precisão, responsável por manter a data e hora do sistema, mesmo quando este está *offline* ou perde a alimentação. Integra uma interface I2C o que torna bastante indicado para a comunicação com microcontroladores, como os ESP32 presentes no nosso projeto. Como acima referido, tem uma precisão muito elevada (erro de cerca de 1 minuto por ano), o que o torna muito mais eficaz que a maioria dos RTC's presentes no mercado.

Este RTC conta com um sensor interno que mede a temperatura ambiente, de forma a poder fazer os ajustes necessários para poder manter a estabilidade temporal independentemente das variações térmicas. Esta característica oferece-lhe uma enorme utilidade em soluções desenvolvidas com o intuito de serem utilizadas no exterior, como a Mobile Heart Box. Tem também uma bateria de *backup* integrada, que lhe permite manter o tempo atualizado em situações de falhas energéticas. Em situações em que está a ser alimentado eletricamente, o seu consumo é muito reduzido, o que o torna ideal para sistemas portáteis. Por fim, a sua capacidade de gerar interrupções e configurar alarmes, permite automatizar tarefas no sistema.

Figura 2.2.6.1: RTC DS3231



2.2.7 Sensor IMU

Um sensor IMU (Fig 2.2.7.1) é um dispositivo capaz de medir o movimento e a orientação de um corpo. É muito utilizado em dispositivos *wearables*, *drones*, *smartphones* e sistemas de navegação. Integra vários sensores, desde acelerômetros (detecção de aceleração e medição da inclinação), giroscópios (capazes de medir a rotação) e magnetômetros (bússola).

A presença de interfaces de comunicação SPI e I2C facilita a integração com microcontroladores ESP32, assim como o seu baixo consumo energético, que é ajustável através de configurações. No entanto, estes sensores precisam de processamento de dados para poderem fornecer informação útil.

Figura 2.2.7.1: Sensor IMU



2.2.8 Sensor mmWave HMMD

O mmWave HMMD (Fig 2.2.8.1) é um sensor de proximidade capaz de identificar presença e movimento. Este sensor faz uso de ondas eletromagnéticas de alta frequência, o que o torna mais fiável que os sensores de proximidade tradicionais. A sua capacidade de atuar mesmo em situações ambientais adversas, como nevoeiro, presença de poeira ou fumo e baixa luminosidade, torna-o bastante útil para um projeto com a Mobile Heart Box, que precisa de funcionar no exterior.

O sensor integra interfaces de comunicação no protocolo UART, o que facilita a sua integração com microcontroladores. Através desta comunicação, é possível configurar vários parâmetros, como o alcance de deteção.

Por fim, o seu reduzido consumo energético e pequenas dimensões fazem com que seja bastante útil em dispositivos móveis.

Figura 2.2.8.1: Sensor mmWave HMMD



2.2.9 Sensor 60GHz mmWave Radar

O Sensor 60GHz mmWave Radar (Fig 2.2.9.1) é um módulo baseado na tecnologia radar FMCW focado na deteção de respiração e batimentos cardíacos, bem como na avaliação do sono. A operar na banda dos 60GHz, este sensor consegue detetar movimentos subtis do corpo. A sua capacidade de funcionar de forma fiável sem ser afetado por fatores ambientais torna-o bastante robusto. Este sensor integra uma interface universal de comunicação no protocolo UART e através desta comunicação, é possível configurar vários parâmetros e extrair dados detalhados em tempo real (taxas de respiração e batimentos por minuto ou o estado de presença).

Este sensor foi apenas utilizado para testes, uma vez que tinha uma limitação que fazia com que perdesse a utilidade no contexto do nosso projeto: só conseguíamos obter medições até aos 100 batimentos por segundo, o que num contexto em que o utilizador está a realizar uma atividade desportiva, seriam regularmente ultrapassados.

Figura 2.2.9.1: Sensor 60GHz mmWave Radar



2.2.10 Sensor AD8232

O sensor AD8232 (Fig 2.2.10.1) é um sensor de frequência cardíaca desenvolvido para recolha de sinais de eletrocardiograma (ECG), através da captação e amplificação dos sinais elétricos produzidos pelo coração.

Este sensor partilha várias das características mais importantes em soluções móveis com os outros sensores acima referidos: baixo consumo de energia, dimensões reduzidas e resistência a cenários adversos.

A medição é efetuada através de 3 elétrodos: um tem de estar à direita do coração, outro à esquerda e o último entre os anteriores. O sinal captado por estes elétrodos é bastante frágil, mas o sensor está dotado com filtros e amplificadores capazes de melhorar a capacidade do sinal antes deste ser enviado para o microcontrolador.

Figura 2.2.10.1: Sensor AD8232



2.2.11 Bluetooth Low Energy

O BLE é uma versão do protocolo de comunicação sem fios Bluetooth, projetada para o baixo consumo energético, tornando-a uma ferramenta muito importante no desenvolvimento de *wearables* e dispositivos IoT. Esse baixo consumo energético verifica-se à custa de um curto alcance e de uma taxa de dados mais baixa que o WiFi, tornando-a ideal para envio de pequenas quantidades de dados, o que se encaixa no uso de sensores como os já referidos ao longo do Capítulo 2.2.

O ESP32 suporta esta versão do protocolo, o que a torna uma solução ideal para enviar dados recolhidos pelos sensores para o smartphone, enquanto contribui para o aumento da autonomia energética do sistema, devido ao seu baixo consumo.

2.2.12 Broker MQTT Eclipse Mosquitto

O Eclipse Mosquitto é um *broker* MQTT de código aberto, responsável por gerir a comunicação entre dispositivos integrados num sistema IoT. Este *broker* atua como intermediário na comunicação entre os dispositivos, seguindo um modelo *publish/subscribe*: um dispositivo publica as mensagens no broker através de um tópico, e outro dispositivo, ao se inscrever nesse mesmo tópico, vai receber as mensagens.

Este broker implementa então o protocolo de mensagens MQTT, caracterizado por um baixo consumo de largura de banda e ideal para comunicações instáveis, daí a nossa preferência por este protocolo para o nosso projeto, pois ao ser uma solução portátil a funcionar no exterior, poderá estar em contextos em que comunicar não seja fácil. Além disso o seu baixo consumo energético é mais um dos motivos pelo qual se enquadra no nosso projeto.

3 Requisitos do sistema e arquitetura

3.1 Requisitos do sistema

Neste capítulo vamos apresentar a especificação dos requisitos do sistema. Os subcapítulos seguintes irão retratar o processo de recolha de requisitos, os requisitos funcionais e não funcionais, atores do sistema e casos de uso.

3.1.1 Recolha de requisitos

O processo de recolha de requisitos focou-se essencialmente na análise de projetos relacionados e do contexto tecnológico na área dos dispositivos IoT e *wearables*. Além disso, realizamos várias entrevistas com possíveis futuros utilizadores, praticantes de *hiking* e *biking*, de forma a perceber o que o utilizador final gostaria de ver numa solução deste género.

O desenvolvimento de casos de uso e *user stories* também nos permitiu perceber que requisitos seriam importantes implementar. Além disso, o facto de termos protótipos simples a funcionar desde uma fase muito precoce do projeto, também nos ajudou nesta recolha. No entanto, e sendo este um projeto bastante exploratório, a nossa recolha de requisitos estará em constante evolução.

3.1.2 Atores

O nosso projeto centra-se num ator principal: um praticante de *biking*. Além deste interveniente principal, realizámos vários testes para praticantes de *hiking*, que apesar de não estarem incluídos na nossa solução, permitiram-nos ter uma ideia do que poderia ser algum do trabalho futuro. O nosso público-alvo é assim todo o praticante desta modalidade que esteja interessado em monitorizar a sua atividade física, assim como em controlar melhor o ambiente que o rodeia. Não necessitam de ser pessoas especializadas em exercício físico ou saúde, pois teremos como objetivo desenvolver uma interface simples de utilizar, em que o utilizador não precise de dominar muitos conceitos tecnológicos e científicos.

É, no entanto, necessário que o utilizador esteja minimamente confortável com o uso de smartphones e consiga compreender que métricas estão a ser lidas pelo nosso dispositivo.

3.1.3 Casos de Uso

Definimos os seguintes casos de uso como base do nosso projeto, que se centram principalmente na interação do utilizador com a aplicação móvel:

1. O utilizador deverá conseguir receber e responder a feedbacks emitidos durante a prática desportiva.
2. O utilizador deverá conseguir visualizar os dados recolhidos pelo sistema em tempo real, no *smartphone* diretamente conectado à Mobile Heart Box.
3. O utilizador deverá conseguir transportar a Mobile Heart Box na bicicleta, para que o dispositivo possa realizar as medições de forma autónoma sem constante necessidade de interação com o utilizador.
4. O utilizador deverá conseguir, depois da atividade, analisar não só os dados da atividade concluída, como poder visualizar estatísticas de atividades anteriores, para poder efetuar comparações e medir o seu progresso.

3.1.4 Requisitos Funcionais

A seguinte lista apresenta os principais requisitos funcionais do sistema, descrevendo as funcionalidades que a Mobile Heart Box deverá disponibilizar.

- Monitorização fisiológica: O sistema deve monitorizar a frequência cardíaca do utilizador em tempo real, permitindo identificar situações de fadiga ou esforço físico excessivo durante a atividade. Prioridade: **alta**.
- Detecção de quedas e imobilidade: Com base nos dados da IMU, o sistema deve distinguir entre paragens normais (por exemplo, semáforos) e eventos críticos como quedas ou estados de imobilidade anormais. Prioridade: **alta**.
- Sistema de alertas automáticos: O sistema deve fornecer *feedback* imediato ao utilizador através de notificações no smartphone sempre que forem detetadas situações críticas. Prioridade: **alta**.

- Rastreamento da localização: O sistema deve ser capaz de rastrear a posição geográfica do utilizador, de forma a poder ilustrar o percurso realizado durante a atividade. Prioridade: **alta**.
- Visualização e configuração de dados: O sistema deve disponibilizar uma aplicação móvel que permita ao utilizador visualizar dados em tempo real e históricos, bem como configurar parâmetros do sistema. Prioridade: **média**.
- Monitorização térmica: O sistema deve realizar a medição contínua da temperatura corporal do utilizador. Prioridade: **média**.
- Monitorização de proximidade: O sistema deve ser capaz de verificar se o utilizador tem algum obstáculo atrás dele, colmatando a falta de espelhos retrovisores na bicicleta. Prioridade: **baixa**.
- Manual do utilizador: O sistema deverá disponibilizar na sua aplicação um manual que permita ao utilizador entender como pode utilizar o dispositivo. Prioridade: **baixa**.

3.1.5 Requisitos Não Funcionais

A seguinte lista apresenta os requisitos não funcionais, ou seja, propriedades e restrições que o sistema deve respeitar para garantir o seu funcionamento adequado em cenários reais.

- Autonomia Energética: Dado que o sistema se destina a utilização em mobilidade, deve ser capaz de operar durante a duração típica de uma atividade física sem necessidade de recarregamento. Devem ser consideradas estratégias de otimização do consumo energético. Prioridade: **alta**.
- Fiabilidade em Mobilidade: O sistema deve garantir a fiabilidade dos dados adquiridos, mesmo na presença de ruído, vibrações e variações ambientais típicas de cenários exteriores. Devem ser aplicadas técnicas de filtragem e processamento de sinal. Prioridade: **alta**.
- Tempo de Resposta: O sistema deve responder rapidamente a eventos detetados, assegurando que os alertas são emitidos com atraso mínimo após a identificação de uma situação crítica. Prioridade: **alta**.

- Portabilidade: O sistema deve ser compacto, leve e de fácil transporte, permitindo a sua utilização confortável em diferentes cenários de mobilidade, como ciclismo ou caminhadas. Prioridade: **média**.
- Usabilidade: A aplicação móvel deve ser simples e intuitiva, permitindo ao utilizador compreender rapidamente a informação apresentada e reagir aos alertas sem distrações durante a atividade. Prioridade: **média**.
- Estética da solução final: A solução final, a nível do hardware (caixa dianteira e traseira), deve ser agradável ao nível estético, aproximando-se de um produto pronto para venda e afastando-se de um protótipo que apenas cumpre com as funções essenciais. Prioridade: **baixa**.

3.1.6 Pressupostos e Dependências

De forma a garantir o funcionamento correto do sistema, são assumidas as seguintes condições e identificadas as principais dependências:

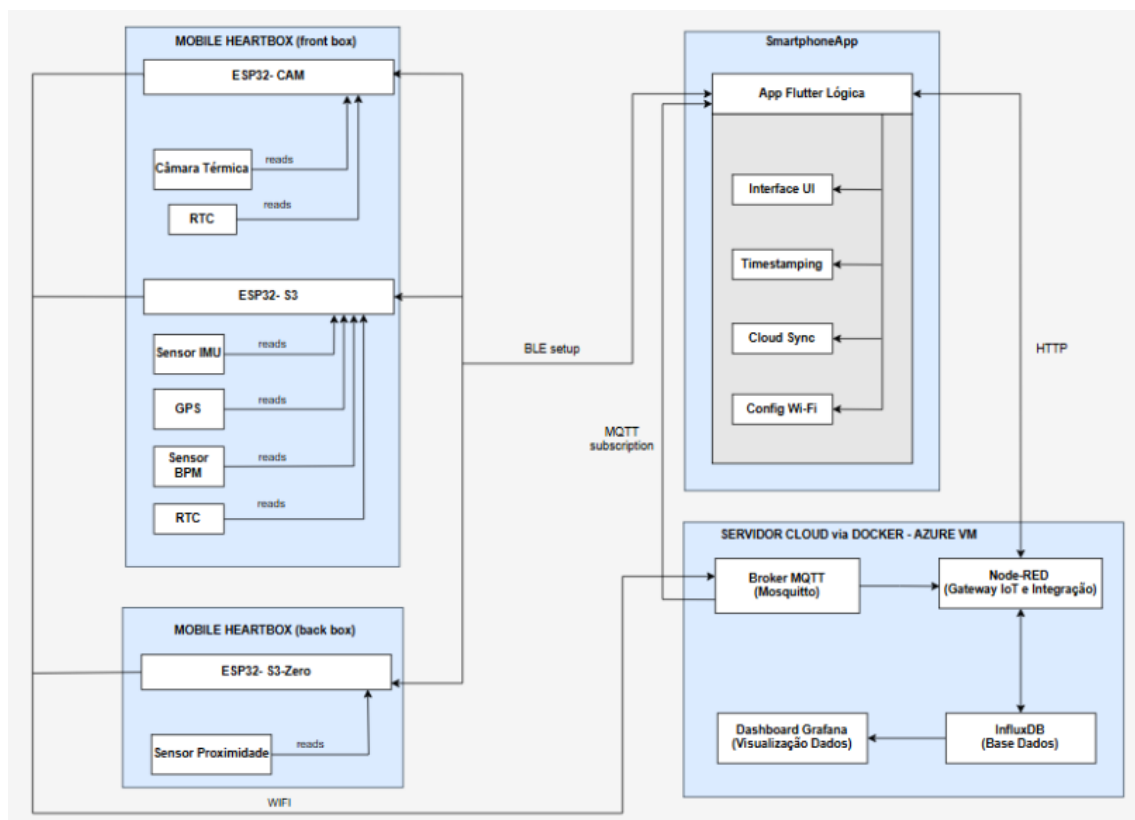
- O sistema pressupõe que o utilizador possui um smartphone compatível com comunicação BLE, necessário para estabelecer a ligação inicial com a Mobile Heart Box.
- Assume-se também que os sensores utilizados (sensor de frequência cardíaca, IMU, sensor de temperatura, sensor de proximidade, módulo de GPS) se encontram corretamente posicionados e calibrados, de modo a garantir a fiabilidade das medições. Um posicionamento inadequado poderá comprometer a fiabilidade dos dados recolhidos.
- Relativamente à transmissão de dados, o sistema depende da ligação WiFi entre o dispositivo e o *smartphone*. Interrupções nesta ligação poderão afetar temporariamente a transmissão de dados em tempo real, embora o sistema deva ser capaz de continuar a operar localmente.
- Adicionalmente, considera-se que o utilizador utiliza o sistema em condições normais de mobilidade (ciclismo), podendo fatores externos como vibrações, condições atmosféricas ou ruído ambiental influenciar o desempenho dos sensores, apesar de, o sistema ser capaz de contrariar os efeitos destes fenómenos em pequena escala.

- De forma a garantir o correto funcionamento do sistema, é necessário que em momentos em que não estão a ser utilizadas, ambas as caixas sejam guardadas de forma segura, de preferência no interior de edifícios. Isto permitirá aumentar o tempo de vida da solução e a redução da probabilidade de avarias a nível do hardware.
- Por fim, o correto funcionamento do sistema depende da autonomia energética do dispositivo, sendo necessário que a bateria esteja devidamente carregada antes do início da atividade.

3.2 Arquitetura do Sistema

Neste capítulo é apresentada a arquitetura do sistema, exemplificada através de um modelo de domínio (Fig. 3.2.1) e de uma explicação dos fluxos de comunicação, incluída na descrição dos componentes.

Figura 3.2.1: Diagrama da Arquitetura de Domínio



Mobile Heart Box (Caixa da frente)

Esta camada é composta pelos microcontroladores ESP32-CAM e ESP32-S3, responsáveis pela aquisição e processamento local dos dados:

- ESP32-CAM: Captura os dados da câmara térmica MLX90640 e utiliza o módulo RTC DS3221 para sincronização temporal dos *frames*. A comunicação com o *smartphone* é feita inicialmente através de BLE, para as configurações iniciais, e posteriormente, através de um broker MQTT, para a transferência de dados.
- ESP32-S3: É integrado com um sensor de batimentos cardíacos, um sensor IMU, um módulo de GPS e um RTC DS3221 (com a mesma função acima referida). Este microcontrolador é responsável por recolher dos sensores acima referidos, que inclui a frequência cardíaca, a deteção de quedas e a localização geográfica. De resto, comunica com o *smartphone* da mesma forma que o ESP32-CAM.

Mobile Heart Box (Caixa de trás)

Esta camada da arquitetura integra apenas o microcontrolador ESP32-S3-Zero, que recolhe os dados enviados pelo sensor de proximidade que se encontra na traseira da bicicleta. Este módulo segue o mesmo modelo de comunicação das outras camadas do sistema, sinalizando a aplicação móvel, através de uma notificação no servidor MQTT, que existe um obstáculo atrás da bicicleta.

Aplicação móvel

A aplicação móvel, desenvolvida em Flutter, funciona maioritariamente como interface com o utilizador, e divide-se nos seguintes componentes:

- Interface UI: camada responsável pela interação com o utilizador.
- Timestamping: responsável por associar uma data e hora aos dados provenientes dos sensores e que são recebidos através do broker MQTT.
- Cloud Sync: envia os dados agregados para a infraestrutura de *cloud*.
- Config WiFi: permite a parametrização da rede WiFi que servirá de base para comunicação entre o Mobile Heart Box e aplicação móvel.

Servidor Cloud via Docker – Azure VM

A camada de *backend* é composta por uma infraestrutura virtualizada baseada em *containers* Docker que correm dentro de uma máquina virtual da Microsoft Azure, que armazena e processa os dados recebidos do *smartphone*:

- Mosquitto (Broker MQTT): Gere a comunicação assíncrona entre o *smartphone* e os microcontroladores, tornando-se fulcral na solução por ser o componente que permite a troca de dados entre estas duas camadas.
- Node-Red: funciona como intermediário entre a aplicação móvel, o broker MQTT e a base de dados. Permite o processamento de dados recolhidos, facilitando a comunicação entre os diferentes componentes.
- InfluxDB: Base de dados de séries temporais para armazenamento eficiente de dados recolhidos pelos sensores
- Grafana: Ferramenta de visualização que gera *dashboards* detalhados para análise dos dados recolhidos.

4 Implementação

Neste capítulo, iremos detalhar de que forma os requisitos recolhidos e arquitetura desenhada se traduziram na nossa solução final.

4.1 Visão geral

Num plano geral, foi possível traduzir todos os requisitos funcionais do sistema em funcionalidades, assim como garantir que o mesmo cumpre com os requisitos não funcionais.

Do ponto de vista do hardware, conseguimos modular duas caixas, através da modelação 3D, que contêm todos os componentes que garantem o funcionamento do sistema. A caixa da frente, é alimentada através de uma powerbank. Os vários componentes desta caixa conectam-se através de ligações elétricas numa placa branca (esquema elétrico no sub-capítulo 4.2.4). Já a caixa de trás, que contêm apenas um microcontrolador e um sensor de proximidade, é alimentada diretamente por uma powerbank de dimensões mais reduzidas. Os microcontroladores presentes nas caixas foram programados de forma a serem capazes de processar os dados provenientes dos sensores e comunicá-los à aplicação móvel.

Do ponto de vista do software, a aplicação móvel foi desenvolvida em Flutter, com a linguagem Dart (desenvolvida pela Google). O servidor de *backend* corre num *container* Docker que está inserido numa máquina virtual da Microsoft Azure. Fazemos uso do InfluxDB, como base de dados com *timestamping*, o que garante a integridade dos dados e permite uma análise posterior à atividade por parte do utilizador. O Node-Red é utilizado como intermediário entre a base de dados, a aplicação móvel e o broker MQTT, simplificando a troca de dados. Por fim, o Grafana é responsável por permitir a visualização dos dados recolhidos através de gráficos.

Todos estes processos serão explicados em maior detalhe nos capítulos seguintes, mas de uma forma geral, podemos garantir que fazem com que o projeto cumpra com os requisitos e a arquitetura definida na fase inicial. São estas as características da nossa solução que fazem com que ela seja capaz de recolher dados provenientes de sensores, funcionar em mobilidade, ter autonomia energética e que tenha uma interface de

comunicação com utilizador simples de utilizar, sem prejuízo da funcionalidade do projeto.

4.2 Hardware

Neste capítulo iremos abordar a forma como a nossa solução foi desenvolvida a nível do hardware.

4.2.1 Caixa dianteira

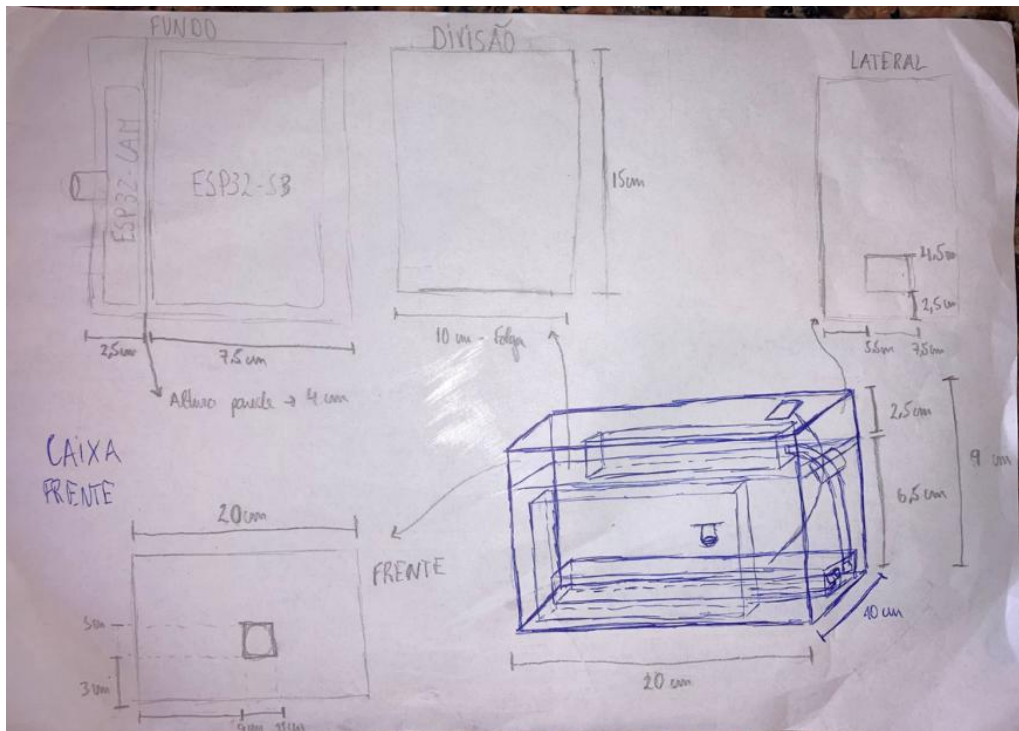
A caixa dianteira foi desenvolvida com recurso à impressão 3D. Essa impressão foi feita com a ajuda de profissionais da Print4Fun3D, localizada em Aveiro, de forma a garantir que se adequava às necessidades do projeto. Tornou-se fundamental arranjar um equilíbrio entre a necessidade de a caixa alojar todos os componentes que a ela estavam designados, e ao mesmo tempo, de garantir que a caixa não reduzia o conforto do ciclista e que não colocava em causa a sua segurança, sendo necessário ter atenção às dimensões e peso da caixa. Além disso, devido à necessidade de ser utilizada no exterior, o material que compõe a caixa devia ser capaz de aguentar fatores ambientais adversos, como chuva ou poeiras, e também sustentar alguma da vibração que surge em alguns dos pavimentos que a bicicleta poderá percorrer.

Dentro desta caixa, encontra-se uma *powerbank*, responsável pela alimentação do circuito, que está montado numa placa branca (esquema elétrico no subcapítulo 4.2.4). A caixa contém também um sensor de batimentos cardíacos, um sensor IMU, um módulo de GPS, um RTC DS3221 e uma câmara térmica, sendo o ESP32-CAM e o ESP32-S3 os microcontroladores responsáveis por fazer o processamento dos dados provenientes destes componentes, assim como realizar algumas configurações necessárias.

Sendo assim, a caixa desenhada teria de conseguir alojar todos estes componentes e as ligações entre eles, deixar aberturas em zonas estratégicas (a câmara térmica por exemplo, precisava de estar apontada para o exterior) e ter algo que a permitisse abrir, de forma a podermos testar diferentes formas de organizações dos componentes e ser possível substituir alguns dos mesmos caso fosse necessário.

Desenhámos então o seguinte esboço (Fig. 4.2.1.1), tendo em conta as necessidades acima definidas, que depois foi entregue aos profissionais (infelizmente, não temos acesso ao desenho utilizado pelos profissionais para imprimir a caixa):

Figura 4.2.1.1: Esboço do desenho da caixa dianteira



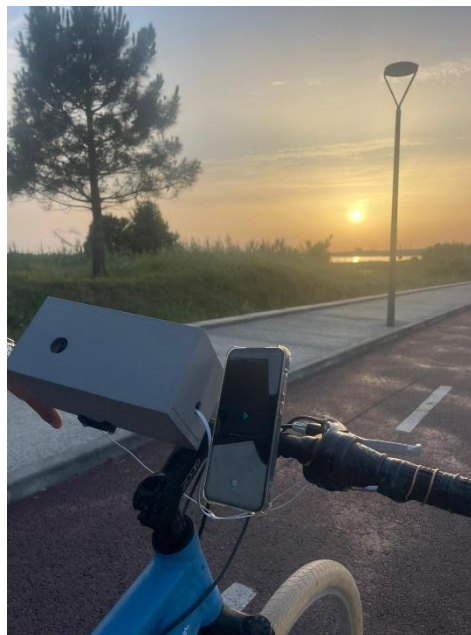
Chamamos a atenção para os seguintes pormenores:

- Criação de um compartimento na parte superior da caixa, junto da porta onde é possível abrir a caixa, onde está colocada a *powerbank*. Este compartimento foi criado pois a *powerbank* será o único componente que estará à responsabilidade do utilizador, pois poderá ter de ser carregada, ou até substituída (para este caso, a caixa contém folga, pelo que se poderá trocar a *powerbank* por uma de maiores dimensões).
- Na parte frontal da caixa, que está virada para o ciclista, existe um orifício destinado à câmara térmica. Isto permite que ela esteja a efetuar leituras da temperatura corporal do ciclista.
- Na parte lateral da caixa (lado direito), existe um orifício destinado à saída dos cabos que conectam o sensor de batimentos cardíacos aos elétrodos colocados no guiador.
- Por fim, e como ficará evidente no resultado da caixa a seguir demonstrado (Fig. 4.2.1.2), foram desenhados pequenos orifícios na parte inferior das laterais da caixa, com a finalidade de prevenir o sobreaquecimento de alguns dos componentes.

Depois da impressão da caixa, foram então colocados os componentes da seguinte forma: *powerbank* no compartimento superior, os cabos de alimentação descem para o compartimento inferior (conectam-se aos microcontroladores), onde estão colocados os restantes componentes; a placa branca fica no fundo da caixa, seguindo uma orientação que permita à câmara térmica e ao sensor de batimentos cardíacos estarem posicionados nos respetivos orifícios.

Por fim, a caixa é encaixada num suporte (comum, encontrado em várias superfícies comerciais, pode ser ajustado pelo utilizador) colocado no guidão da bicicleta, com a câmara térmica virada para o utilizador e os cabos do sensor de batimentos cardíacos a saírem pelo lado direito. É também colocado na parte superior da caixa o QR Code que permite a comunicação via BLE com a aplicação móvel. A Figura 4.2.2.2 ilustra então como a caixa se integra com a bicicleta:

Figura 4.2.1.2: Integração da caixa na bicicleta



4.2.2 Caixa traseira

O processo de desenvolvimento da caixa traseira foi bastante semelhante com o da caixa dianteira, tendo apenas algumas diferenças no seu desenho, devido à necessidade de alocar componentes diferentes.

Como referido na arquitetura, na caixa traseira estão apenas contidos um microcontrolador ESP32-S3-Zero e um sensor de proximidade, assim como uma

powerbank (com dimensões mais reduzidas que a da caixa dianteira) para alimentação. O tamanho pode então ser mais reduzido em relação à caixa dianteira, necessitando apenas de um orifício na parte de trás da caixa para que o sensor de proximidade consiga detetar obstáculos. Além disso, a caixa também conta, tal como a dianteira, com pequenos orifícios para ventilação, uma porta para possibilitar a abertura e um compartimento para a *powerbank*.

Por fim, a caixa é a montada na bicicleta, debaixo do selim, com um suporte universal igual ao que é colocado no guiador para a caixa dianteira. É importante que o orifício onde está posicionado o sensor de proximidade se encontre virando para a traseira da bicicleta e ligeiramente inclinado para cima, de forma a não indicar o pneu traseiro da própria bicicleta como um obstáculo. Nas seguintes imagens mostramos os esboços do desenho (mais uma vez não temos acesso ao desenho técnico) e o resultado da caixa:

Figura 4.2.2.1: Esboço do desenho da caixa traseira

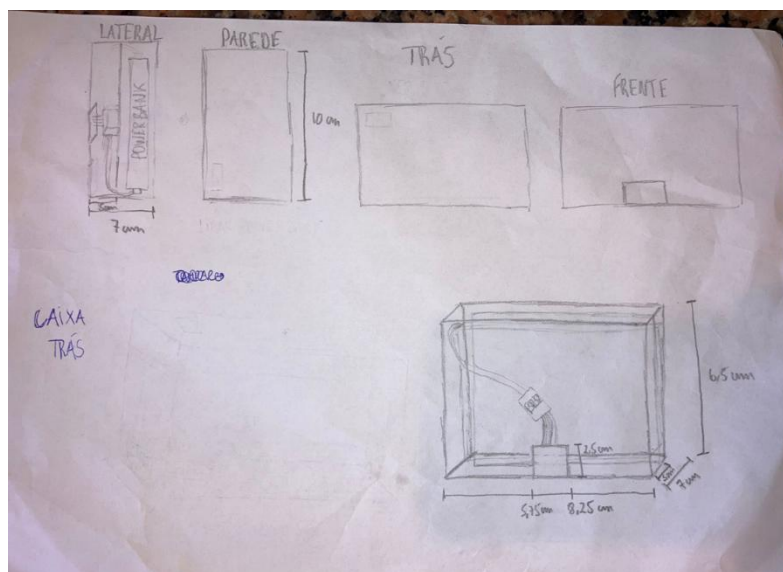


Figura 4.2.2.2: Caixa traseira montada na bicicleta



4.2.3 Integração de sensores

Como referido nos capítulos anteriores, os sensores estão distribuídos por 3 microcontroladores. De seguida, apresentamos de que forma os diferentes sensores devem ser integrados com os respetivos microcontroladores, e posicionados dentro da caixa, de forma a garantir um bom funcionamento do sistema:

- ESP32-CAM: está conectado a uma câmara térmica MLX90640 e a um RTC DS3221. A câmara térmica, após conectada com o microcontrolador (esquemas elétricos especificados no capítulo 4.2.4), terá de ser enquadrada com o orifício correspondente. O RTC não tem necessidades de posicionamento específicas, sendo apenas colocado numa posição que facilite a organização dos componentes.
- ESP32-S3: contém o sensor IMU, o módulo de GPS, o sensor de batimentos cardíacos e outro RTC DS322 (mesmas orientações do ponto anterior). O sensor o IMU e o módulo de GPS precisam de estar numa posição horizontal, por motivos diferentes: o primeiro necessita de estar numa posição correspondente aos 0°, ou muito perto disso, pois deteta quedas quando ultrapassa os 60° de inclinação lateral; já o segundo, necessita de estar apontado para cima de maneira a facilitar a comunicação satélite. Por último, o sensor de batimentos cardíacos, deverá ser colocado de forma que permita que os cabos conectados aos elétrodo saírem pelo

lado direito da caixa. Para que o utilizador não tenha que estar sempre em contacto com os eléctrodos, e de forma que os cabos não atrapalhem a condução da bicicleta, os eléctrodos são posicionados no início dos manípulos presentes no guiador, e colocados em contacto direto com um fio de cobre, que se envolve no restante manípulo. Isto permite que as ligações sejam mais fidedignas, uma vez que o sensor efetua as suas medições através de impulsos elétricos, que o cobre por ser um excelente condutor, consegue propagar, evitando a necessidade de contacto constante entre o utilizador e os eléctrodos.

- ESP32-S3-Zero: este microcontrolador necessita apenas de comunicar com o sensor de proximidade. O sensor necessita de estar diretamente encaixado no seu orifício correspondente, de maneira a garantir uma deteção fiável de obstáculos.

4.2.4 Esquemas elétricos

Os esquemas elétricos apresentados nesta secção representam as ligações e componentes utilizados no sistema desenvolvido. O sistema é assim composto por duas unidades físicas instaladas na bicicleta - uma caixa na parte frontal e outra na parte traseira, sendo cada uma equipada com os respetivos sensores. Para a caixa que está na parte da frente da bicicleta foram utilizados para além da powerbank uma ESP32-S3 e uma ESP32-CAM com os respetivos sensores associados a cada uma das ESPs. Já para a caixa que está na parte de trás foi utilizado para além de outra powerbank, uma ESP32-S3-ZERO com os sensores associados a essa mesma ESP.

A parte da frente vai assim estar subdividida em dois esquemas (um para a ESP32-S3 com os sensores de batimentos cardíacos, sensor de deteção de quedas e o sensor de gps e depois outro para a ESP32-CAM com o rtc e o sensor responsável pela medição da temperatura corporal). Por sua vez, a parte de trás vai estar apenas representada num esquema (um para a ESP32-S3-ZERO com o sensor de deteção de obstáculos próximos).

Para além dos esquemas do sistema final, são também apresentados nesta secção dois esquemas relativos a dois sensores adicionais que foram testados durante o desenvolvimento do projeto, mas que não foram integrados na versão final. Estes dois sensores estão ambos relacionados com a parte dos batimentos cardíacos e não foram

integrados devido a terem sido encontradas algumas nuances aquando da realização de testes sobre estes sensores.

Figura 4.2.4.1: Esquema da ESP32-CAM relativo à parte frontal da bicicleta

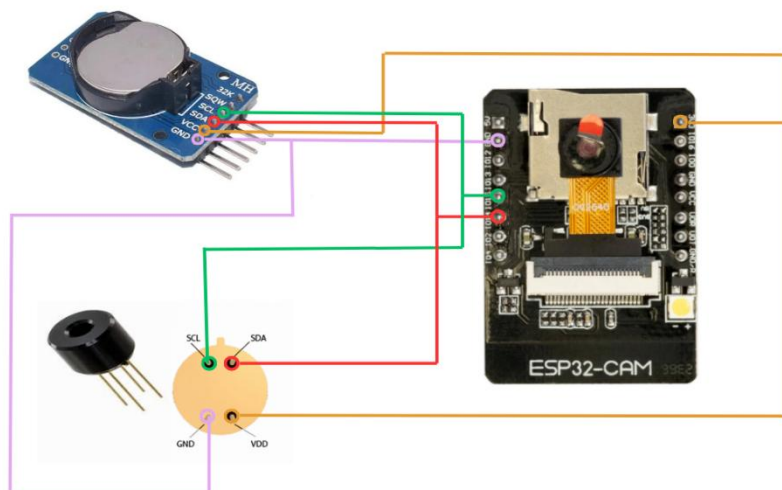


Figura 4.2.4.2: Esquema da ESP32-S3 relativo à parte frontal da bicicleta

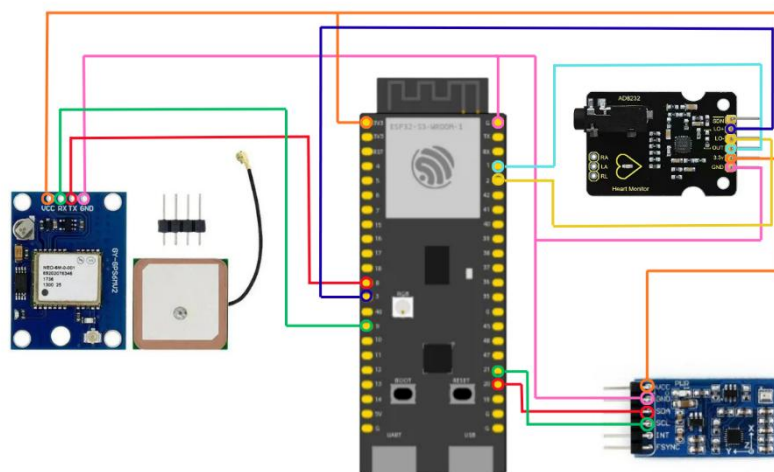


Figura 4.2.4.3: Esquema da ESP32-S3-ZERO relativo à parte traseira da bicicleta

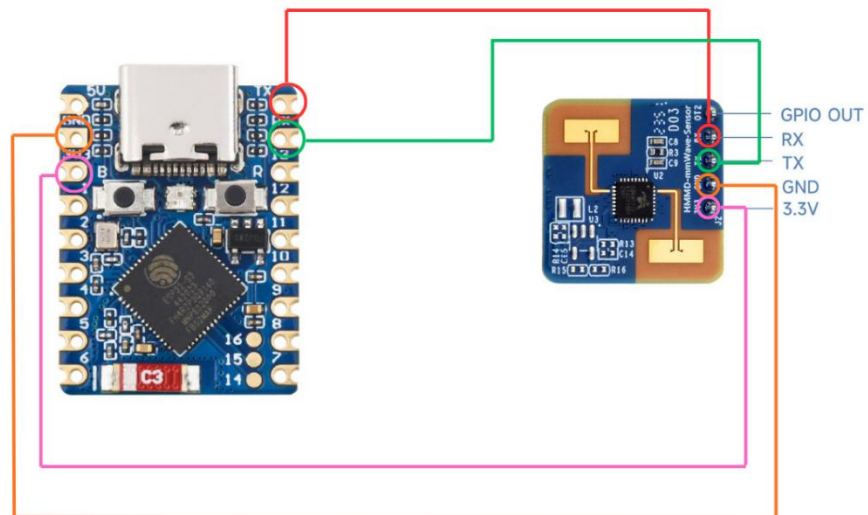


Figura 4.2.4.4: Esquema da ESP32-S3 integrada com o sensor PPG (não usado)

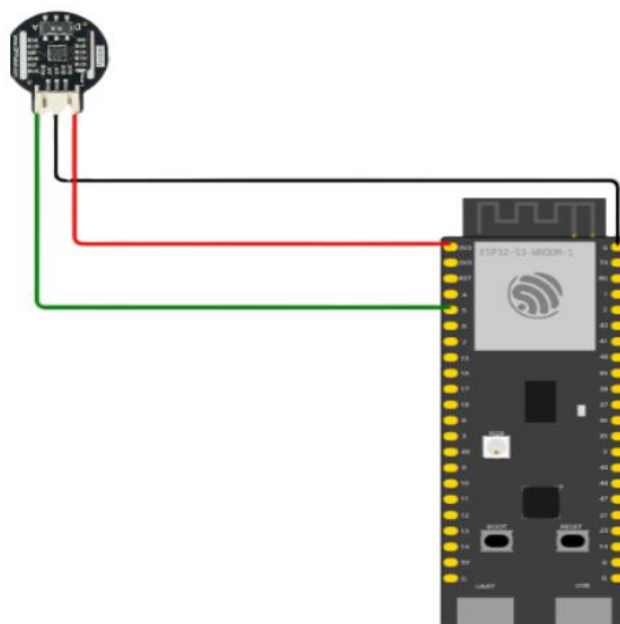
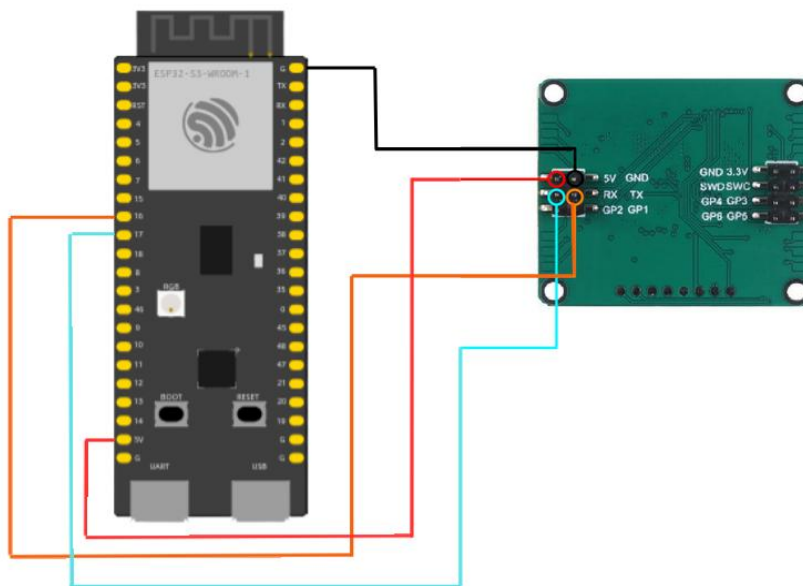


Fig 4.2.4.5: Esquema da ESP32-S3 integrada com o sensor R60A (não usado)

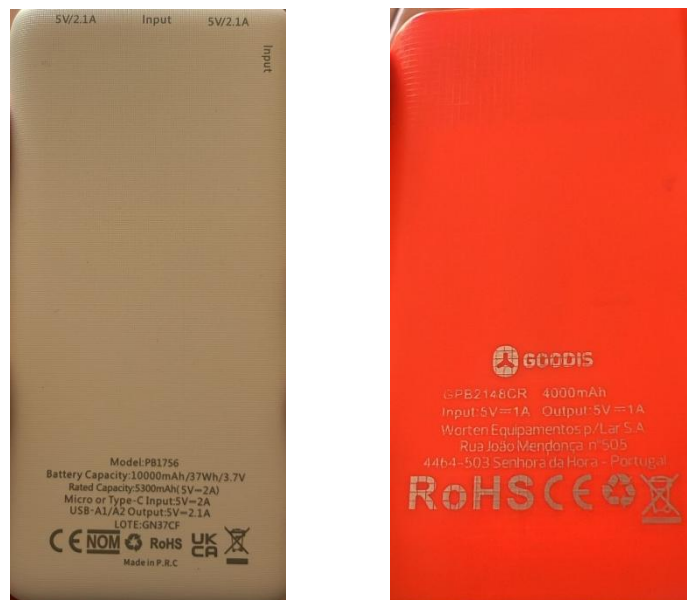


4.2.5 Alimentação e gestão energética

A alimentação é feita por duas *powerbanks*, uma em cada caixa, que estão alocadas num compartimento específico para que seja possível ao utilizador trocá-las (desde que as dimensões sejam adequadas ao sistema) em caso de avaria, ou se pretender obter uma maior autonomia.

A *powerbank* da caixa dianteira tem no total 37Wh armazenados quando completamente carregada. Uma *powerbank* desta capacidade oferece ao sistema uma autonomia de 12 horas, pelo que se pode concluir que os componentes presentes na caixa dianteira consomem energia a cerca de 3W. Já a da caixa da traseira, tem uma capacidade de 15Wh, oferecendo a mesma autonomia que a caixa dianteira. Podemos então concluir que os componentes presentes na caixa traseira consomem energia a cerca de 1W. Com estas informações, o utilizador pode adaptar a *powerbank* ao nível de autonomia que pretende.

Figura 4.2.5.1: *Powerbanks* presentes no sistema



4.3 Firmware

Neste capítulo, iremos explorar o *firmware* desenvolvido para cada microcontrolador presente no nosso projeto.

4.3.1 ESP32-CAM

A função `setup()` é a primeira a correr quando a placa arranca. A primeira operação que esta função faz é verificar se existem credenciais de WiFi na memória persistente da placa. Caso não existam, é arrancado o modo de configuração via BLE. Independentemente da existência de credencias WiFi, esta função inicializa todo o hardware que está ligado à placa (RTC e câmara térmica), ou seja, mesmo que a placa se encontre a ser configurada, a câmara térmica encontra-se operacional.

Após a receção das credenciais, o *firmware* desenvolvido faz as verificações necessárias para garantir que o SSID e a *password* foram indicados, e que estão corretos (as verificações podem falhar caso a rede não esteja disponível ou a *password* esteja incorreta). Se as credenciais estiverem corretas, é agendado um *reboot* da placa para aplicar essas configurações e as credenciais são guardadas na memória persistente da placa. Após o setup terminar, o modo de operação fica definido como *Running_WiFi*, sendo a única forma de regressar ao BLE, reiniciar a placa.

O modo *Running_Wifi* corre então dentro de um *loop* infinito, com uma estratégia de reconexão caso a ligação com o servidor MQTT caia. Neste caso, a função `setup_wifi()` é chamada de forma a tentar restabelecer a ligação, sendo o intervalo entre tentativas cada vez maior (até atingir um limite). Quando a ligação está funcional, é chamada a função `mqttCliente.loop()`, que envia os dados a cada 0,5 segundos. Para evitar colisões, os identificadores de cada cliente são gerados aleatoriamente. Os dados são enviados através da publicação de mensagens em dois tópicos do *broker* MQTT (o tamanho do *payload* foi aumentado no código): no *heartbox/sensor/termal*, a temperatura máxima recolhida pelo sensor, como *string*; e no tópico *heartbox/cam/thermal_raw* a matriz térmica em formato *raw*.

Os dados são obtidos da câmara térmica configurando-a para trabalhar no modo *chess*, em que os pixéis são distribuídos de forma semelhante a um tabuleiro de xadrez e lidos de forma alternada, num *frame* são lidos os pixéis nas posições brancas e no seguinte

são lidos os pixels nas posições pretas. A resolução no ADC é de 18 bits (máximo do sensor) e com uma frequência de *refresh* de 2Hz, longe do limite de 64Hz, mas que se adequa ao intervalo com que os dados são enviados através do *broker* MQTT.

A função que recolhe os dados provenientes da câmara térmica implementa uma lógica de recuperação, caso não consiga extrair os mesmos. Cada tentativa tem um timeout de 1 segundo (o sensor é sondado cerca de 100 vezes por tentativa). É respeitado um delay entre tentativas (vai crescendo em 0,1 segundos, começando nesse mesmo valor) o que permite ao barramento estabilizar antes de tentar de novo. Antes de iniciar a terceira tentativa, a frequência de *refresh* é reduzida para 1Hz para aumentar a probabilidade de obter sucesso na recolha de dados. Após 5 erros consecutivos, é invocada a função *setup_sensor()*, que reinicia o sensor sem ser necessário dar *reboot* a toda a placa.

Caso as leituras consigam ser efetuadas, o *firmware* faz também algum do processamento dos dados recolhidos. Todos os valores fora do intervalo de -40 a 300 graus Celsius são descartados, por indicação do fabricante, pois indicam valores fisicamente impossíveis de serem recolhidos pelo sensor, sugerindo uma possível corrupção na recolha dos dados (esses pixels são marcados com uma *flag* de erro). Após este processamento, os dados são transmitidos da forma acima referida.

4.3.2 ESP32-S3

Esta placa contém o firmware mais complexo por integrar um maior número de sensores: módulo de GPS, sensor IMU e sensor de batimentos cardíacos. Quanto aos modos de operação, o funcionamento é semelhante ao detalhado no subcapítulo anterior: tenta ler as credenciais WiFi da sua memória persistente, e se não for possível inicia a configuração via BLE. Os 3 sensores correm dentro do mesmo *loop* (*process_sensors*).

O módulo GPS comunica com a placa via UART, configurada com um *baudrate* de 9600 *bps*. As mensagens são enviadas através do protocolo de mensagens NMEA, sendo a biblioteca TinyGPS++ responsável por fazer o *parsing* dessas mensagens. O resultado deste *parsing* é publicado no servidor MQTT a cada 2 segundos (tópico *heartbeat/gps/coords*), com formato “latitude,longitude”. Quando o módulo de GPS não consegue recolher coordenadas válidas, publica a mensagem “Sem sinal de GPS”, que

necessita de processamento diferente das restantes mensagens, por parte da aplicação móvel.

A biblioteca *Waveshare_10Dof-D* é responsável por gerir o sensor IMU. A função *processing_sensors* recolhe os ângulos já processados pela biblioteca. O sensor utiliza uma interface I2C para comunicar com a placa. Para a leitura de batimentos cardíacos, é primeiro realizada uma verificação do estado dos elétrodos. Caso estes estejam soltos, é publicada uma mensagem de erro “!”, indicando que não está a ser possível realizar medições, e é limpo todo o estado do algoritmo de leitura dos batimentos cardíacos. Isto faz com que leituras efetuadas enquanto os elétrodos não estão funcionais não sejam interpretadas como batimentos cardíacos.

Uma vez que o sinal elétrico do coração medido na superfície da pele tem uma amplitude demasiado pequena para ser lida diretamente por um ADC, o sensor de batimentos cardíacos trata de amplificar o sinal antes deste chegar ao microcontrolador. O ADC do ESP32-S3 tem uma resolução de 12 bits, mas o *threshold* definido é 2000, significando que o pico de um impulso elétrico recebido pelos elétrodos só é considerado um batimento cardíaco se ultrapassar este valor. Para detetar picos, o código implementa uma lógica que compara estados consecutivos. Quando o sinal está em crescendo, o código determina o valor máximo do pico, até o sinal começar a descer (como o sinal tem um formato de onda, é quando o sinal atual for menor que o anterior). Se esse valor máximo for maior que *threshold*, é validado. A *flag inPeak* bloqueia novos picos enquanto o valor do sinal não descer do *threshold* em 200 unidades, evitando disparos duplos em picos com medições irregulares no seu topo. Em condições ideais, são retiradas cerca de 100 amostras por segundo.

A cada pico validado, a função *calculateBPM()* calcula o tempo que passou desde o último pico. Se esse valor for menor que 0,4 segundos, é descartado, por corresponder a valores de batimentos por minutos fisicamente impossíveis. O mesmo é feito com valores que 2,5 segundos. Os valores dos intervalos validados são então armazenados num array circular de 8 posições, descartando os primeiros 4 valores, que normalmente correspondem a um intervalo de calibração. Através desse *array* circular, é calculada a mediana (sem alterar o *array* original), por ser mais robusta contra *outliers*. Após o cálculo da mediana, é calculado o desvio de cada medição original a este valor da mediana, caso esse desvio seja menor que 25% o valor é considerado clinicamente relevante e usado para calcular os batimentos cardíacos (faz-se a média de

todos os valores clinicamente relevantes). Valores de batimentos por minuto fora do intervalo de 45 a 150 são descartados, mantendo a variável com bpm com o último valor correto. Os valores são publicados no *broker* MQTT a cada 5 segundos.

Para a detecção de quedas, é lido em cada iteração do *loop* infinito os valores do *roll* e do *pitch* (inclinação lateral e frontal). Se algum destes valores ultrapassar os 60°, é sinalizada uma queda através do *broker* MQTT (tópico *heartbox/alerts/fall*). A *flag fallDetected* garante que cada evento de queda é publicado apenas uma vez, evitando o *flooding* do broker. O alerta é publicado fora do intervalo normal, imediatamente após a sua detecção, devido à sua urgência.

O *firmware* conta também com um mecanismo de *keep alive*, que faz com que caso não haja comunicações durante 15 segundos, o broker considere que a ligação está perdida. Este valor de *timeout* é mais pequeno que o usual devido à necessidade de transmitir dados cruciais, como a detecção de quedas.

4.3.3 ESP32-S3-Zero

A ESP32-S3-Zero é o microcontrolador presente na caixa traseira, tendo como única responsabilidade a leitura e comunicação dos dados provenientes do sensor de proximidade mmWave HMMD. A comunicação com este sensor é feita através de um porto série hardware (*HardwareSerial*) nos pinos RX (4) e TX (5), inicializado a 115200 baud.

A função *setup()* é a primeira a correr quando a placa arranca. Tal como na ESP32-CAM, a primeira operação consiste em verificar se existem credenciais de WiFi armazenadas na memória persistente da placa através do *Preferences*. Caso não existam, é iniciado o modo de configuração via BLE, onde a placa fica a aguardar a receção das credenciais através das características definidas para o SSID, password e IP do broker MQTT. Após a receção do IP do servidor, as credenciais são guardadas na memória persistente e é agendado um reboot da placa ao fim de 5 segundos para aplicar as configurações.

O modo de operação principal corre dentro de um loop infinito, com verificação contínua do estado da ligação WiFi e MQTT, recorrendo às funções *setup_wifi()* e *reconnect()* respetivamente sempre que necessário. À semelhança das outras placas, o identificador do cliente MQTT é gerado aleatoriamente para evitar conflitos.

A leitura do sensor mmWave é feita linha a linha através do porto série. O firmware interpreta duas situações distintas:

- Quando a linha recebida começa por *Range*, é extraído o valor numérico da distância em centímetros. Se essa distância for inferior ou igual ao limite definido de 100 cm e ainda não tiver sido enviado um alerta, é publicada no tópico *heartbox/sensor/proximity* a mensagem *OBSTACULO_PERTO*. Quando a distância volta a superar o limite, é publicada a mensagem *CAMINHO_LIVRE* e o estado de alerta é repostado.
- Quando a linha recebida é *OFF*, significa que o sensor não deteta qualquer presença. Nesse caso, se existia um alerta ativo, é igualmente publicada a mensagem *CAMINHO_LIVRE*.

A variável booleana *alertaEnviado* funciona como flag de controlo, garantindo que as mensagens MQTT só são publicadas quando existe uma efetiva mudança de estado, evitando publicações redundantes e desnecessárias no broker.

4.4 Aplicação móvel

Neste subcapítulo iremos abordar o desenvolvimento da aplicação móvel utilizada no nosso projeto.

4.4.1 Flutter

O desenvolvimento da aplicação móvel da Mobile Heart Box foi realizado em Flutter, um *framework open-source* da Google baseado na linguagem de programação Dart. A escolha deste *framework* revelou-se adequada ao nosso projeto pelas seguintes razões:

- **Natureza *cross-platform*:** permite gerar uma única base de código para Android e iOS, garantindo um bom funcionamento da aplicação em diferentes plataformas.
- **Alto desempenho:** o Flutter compila diretamente o código em Dart para a linguagem nativa do dispositivo. Além disso, não depende do sistema operativo para correr a interface, pois faz uso do Impeller (um motor gráfico de alto desempenho). Isto resulta numa execução muito mais rápida e eficaz a nível do CPU, GPU e memória, tornando o Flutter ideal para projetos que necessitam de atualizações frequentes da interface, como a Mobile Heart Box.
- **Hot reload:** as alterações feitas no código são imediatamente refletidas na aplicação, sem necessidade de reiniciar a mesma ou de perder o estado atual. Esta característica facilita o processo de desenvolvimento e testagem da aplicação.
- **Disponibilidade de *packages*:** a grande quantidade de *packages* que o Flutter disponibiliza facilita a integração de diversos protocolos na aplicação sem a necessidade de escrever código nativo muito extenso.

4.4.2 Arquitetura da aplicação

A aplicação do Mobile Heart Box segue uma arquitetura baseada em ecrãs, muito comum em aplicações desenvolvidas no Flutter. Os ecrãs são uma extensão da classe *StatefulWidget*, indicada para interfaces dinâmicas que precisam de atualizar o seu estado ao longo do tempo.

A navegação é feita essencialmente através do *Navigator* (funciona como uma *stack* de ecrãs) e do *MaterialPageRoute* (define qual ecrã deve ser apresentado e a forma com que esse mesmo ecrã é exibido). Os ecrãs principais da aplicação são os seguintes:

- **HomeScreen:** ecrã inicial com o *branding* e onde se efetua o acesso ao sistema (Fig 4.4.2.1).
- **ActivityMenuScreen:** ecrã central com *BottomNavigationBar* de 3 abas, onde estão os acessos às configurações, o dashboard (ou o início à atividade) e o histórico (Fig 4.4.2.1).
- **ActivityScreen:** seleção do tipo de atividade, neste momento só está disponível a bicicleta, mas futuramente o projeto deverá funcionar também em contexto de caminhada (Fig 4.4.2.1).
- **MonitorScreen:** ecrã principal da monitorização em tempo real. Contém as medições da temperatura (visualização térmica), batimentos cardíacos, o mapa e o controlo da atividade (Fig 4.4.2.2).
- **ActivityDetailScreen:** visualização de uma atividade já concluída (Fig 4.4.2.2).
- **QRScannerScreen:** ecrã para leitura do QR Code presente na caixa dianteira.
- **WifiConfigScreen:** ecrã que permite inserir as credenciais do ponto de acesso que se pretende utilizar (Fig 4.4.2.2).

Figura 4.4.2.1: Ecrã inicial, *Dashboard* e ecrã de seleção da atividade

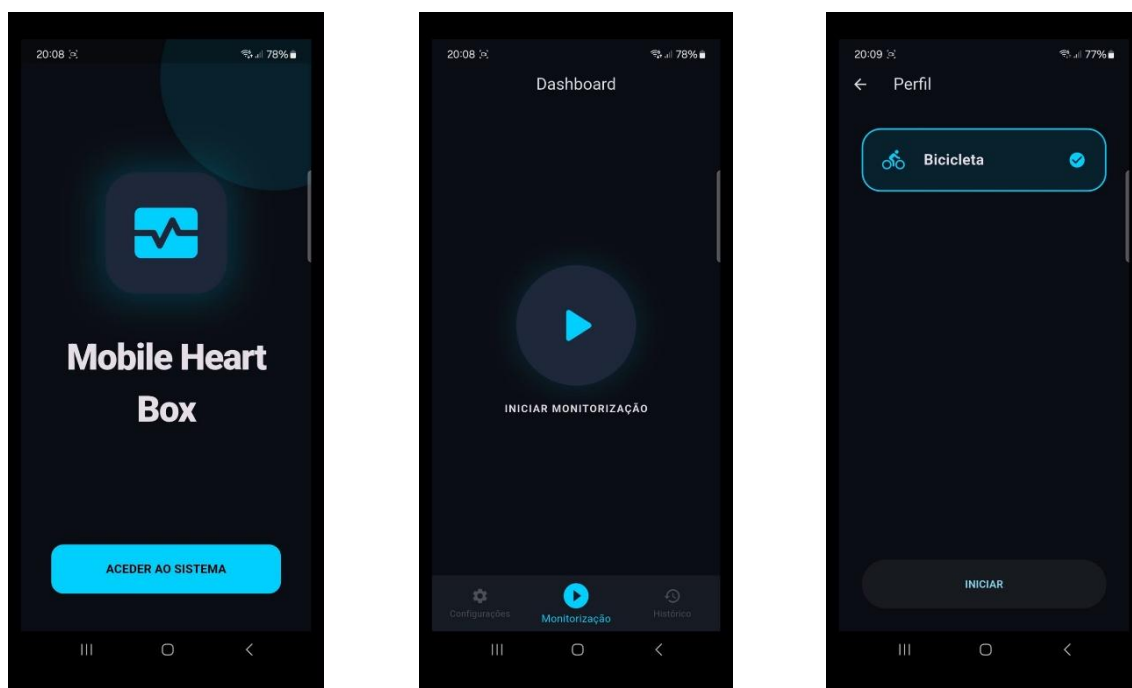
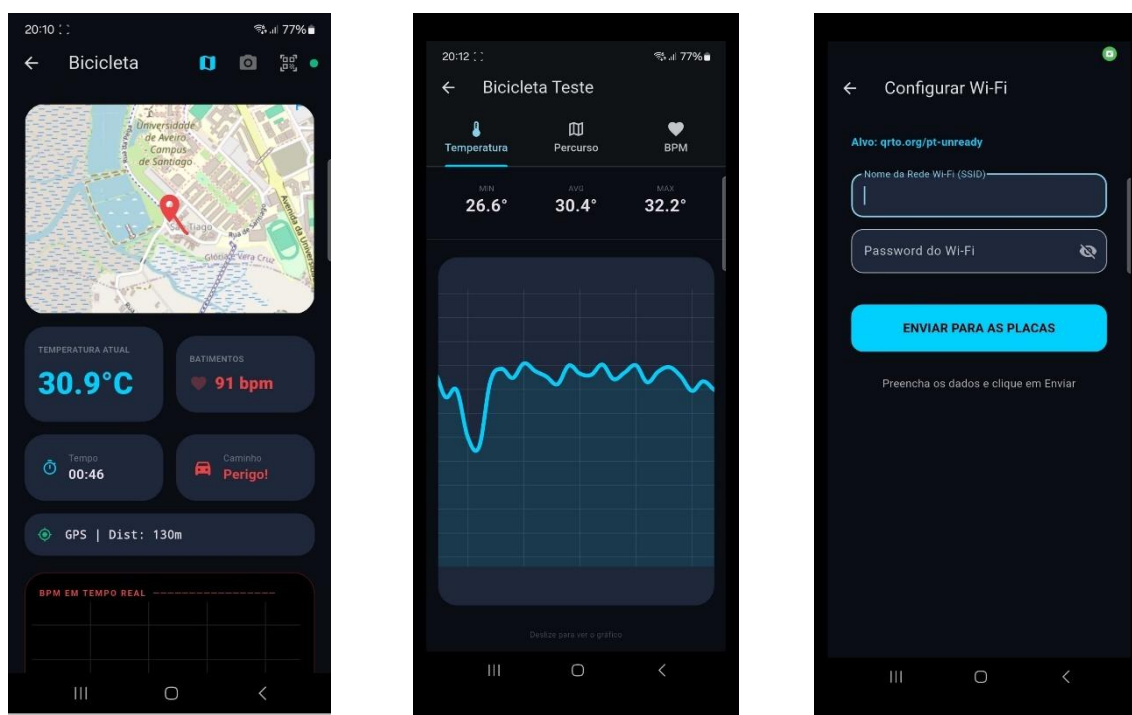


Figura 4.4.2.2: Ecrã de monitorização em tempo real, histórico e configuração WiFi



A aplicação faz também uso dos seguintes pacotes:

- **mqtt_client/mqtt_server_client**: para comunicação em tempo real com o *broker* MQTT.
- **flutter_map/latlong2**: para desenho dos mapas e rota do GPS.
- **fl_chart**: criação de gráficos relativos aos batimentos cardíacos e temperatura.
- **flutter_blue_plus**: comunicação via BLE para a configuração do ponto de acesso
- **mobile_scanner**: leitura do QR Code.
- **shared_preferences**: garante a persistência dos dados (subcapítulo 4.4.3).
- **http**: para comunicação com o Node-Red.
- **url_launcher**: possibilitam a realização de chamadas de emergência.
- **flutter/services**: funcionalidades nativas.

4.4.3 Gestão de estado

A classe *SessionManager* funciona como um gestor persistente do estado do sistema. Todos os membros desta classe são *static*, fazendo com que ela tenha um funcionamento semelhante a um *singleton*. Esta classe faz uso do *SharedPreferences*, um método de armazenamento da linguagem Dart que permite guardar pequenas quantidades de dados de forma persistente no dispositivo (fica no armazenamento, se a aplicação for encerrada, os dados não são apagados). Este tipo de armazenamento segue uma lógica de chave-valor, e permite que os dados relativos a uma atividade sejam mantidos caso a mesma não seja encerrada corretamente. O *SessionManager* gere todos os dados recolhidos pelos sensores, assim como informações relativas à atividade. As suas principais funções são as seguintes:

- **init()**: carrega todos os dados guardados ao iniciar a app. Se a app tiver sido encerrada sem a sessão ter terminado, esta função encaminha o utilizador diretamente para o *MonitorScreen*.
- **start(String type)**: inicia uma nova sessão do tipo passado como argumento, limpando os dados de sessões anteriores.
- **saveProgress()**: guarda o estado atual dos dados presentes no *SharedPreferences* criado, é chamada de segundo em segundo, sendo fulcrar para manter a persistência dos dados.

- **clear():** apaga todos os dados de uma sessão após a mesma ter sido concluída.

Além do *SessionManager*, e como referido anteriormente, cada ecrã tem uma classe *State* (extensão de *StatefulWidget*) onde são declaradas as variáveis do ecrã que são passíveis de uma mudança de estado. Sempre que é necessário atualizar uma dessas variáveis, a função *setState* é chamada. Esta função, além de atualizar as variáveis, notifica também o Flutter para que este possa reconstruir as partes da interface que são afetadas pela mudança de estado.

Para a gestão de estados, a aplicação móvel faz também uso de *timers* e subscrições nos tópicos (do *broker* MQTT) correspondentes a cada sensor integrado no sistema.

4.4.4 Comunicação com o broker MQTT

Como poderá ler no subcapítulo 4.5.1, o *broker* MQTT corre numa máquina virtual do Microsoft Azure, sendo o seu *ip* o indicado em *SessionManager.serverIP*. A aplicação móvel atua apenas como *subscriber*, recebendo apenas dados em tempo real sem publicar mensagens. A ligação é feita através do método *connectMQTT()* presente na classe *MonitorScreenState*.

O cliente é instanciado a partir da função *MqttServerClient* (biblioteca *mqtt_client*), gerando um *clientID* dinamicamente com base no *timestamp* atual, garantindo unicidade caso várias instâncias da aplicação estejam a correr ao mesmo tempo. A *flag autoReconnect* é também colocada como verdadeira, delegando ao cliente a responsabilidade de se reconectar ao servidor em caso de falha.

Após se conectar ao servidor com sucesso, o cliente subscreve-se aos seguintes tópicos, com QoS 0 (a perda ocasional de dados é tolerável):

- **heartbox/sensor/termal:** temperatura real em graus Celsius. A app valida se é menor que 50°C, incrementa a variável *allTemps* com esse valor e exibe-o em tempo real.
- **heartbox/cam/termal_raw:** matriz térmica *raw* de 32x24 pixéis. É depois processada pelo *ThermalPainter*.

- ***heartbox/gps/coords:*** coordenadas GPS. A app calcula depois a distância incremental e guarda as coordenadas numa lista que corresponde à rota percorrida.
- ***heartbox/heart/bpm:*** batimentos cardíacos por minuto. É recebido o carácter “!” se houver algum erro na recolha efetuada pelos elétrodos. Os primeiros 3 valores recebidos são ignorados como estratégia de calibração.
- ***heartbox/alerts/fall:*** alerta de queda. Dispara o serviço de emergência.
- ***Heartbox/sensor/proximity:*** alerta de obstáculo atrás da bicicleta. Aciona o aviso de perigo presente na interface.

Na maioria dos tópicos, o payload recebido é convertido para String antes de se aplicar a lógica correspondente. A única exceção é o tópico da matriz térmica, em que o *array* de *bytes* recebido é diretamente passado para a classe responsável pelo processamento da matriz.

Além da reconexão automática, a aplicação implementa um monitor de rede independente baseado num timer periódico (iniciado no método *startNetworkManager*). A cada 2 segundos é verificado o estado da ligação ao servidor. Se a ligação estiver a falhar, é incrementado o contador *disconnectSeconds*. Enquanto este contador não ultrapassa os 120 segundos, é exibido um aviso a informar que o cliente se está a tentar reconectar ao servidor, confiando na reconexão automática (caso tenha sucesso, coloca o contador a zero). Se esses 120 segundos forem ultrapassados, o sistema declara erro fatal, indicando ao utilizador que deve inserir novas configurações via QR Code. O cliente e todos os *timers* são libertados no método *dispose()*.

4.4.5 Ecrã de monitorização da atividade

Nos próximos 3 subcapítulos, entraremos em maior detalhe no funcionamento de 3 ecrãs, que devido à sua complexidade e importância no funcionamento do sistema, merecem maior atenção.

O *MonitorScreen* é o ponto central da aplicação, pois é o ecrã onde o utilizador está durante toda a atividade física. O *layout* foi desenhado de forma a ser legível em movimento e de maneira a não ser necessária fazer *scroll* para poder visualizar todos os dados importantes para a atividade (o utilizador apenas necessita de fazer *scroll* se quiser visualizar o gráfico dos batimentos cardíacos). Como se pode ver na Fig 4.4.5.1, o ecrã é composto pelos seguintes componentes:

- Na parte superior, indica o tipo de atividade e conta com 3 ícones. Os 2 primeiros permitem alternar entre a visualização do mapa e da matriz térmica (ambos estes componentes são substituídos por uma mensagem de erro caso não consigam ser exibidos). O último permite ler um QR Code caso se pretenda alterar o ponto de acesso. Conta ainda com um indicador do estado da conexão ao *broker* MQTT.
- O ecrã exibe lado a lado a temperatura atual do ciclista e os batimentos cardíacos do mesmo. Quando a sessão é pausada, a temperatura é exibida a branco e o rótulo avisa que a mesma não está a ser atualizada. A secção dos batimentos cardíacos, exibe uma mensagem específica em caso de erro na recolha dos dados e outra mensagem para avisar o utilizador que o sensor ainda se encontra em período de calibração.
- Diretamente por baixo destas secções, são exibidos o tempo de atividade e o indicador de obstáculos, que altera entre a cor verde e a mensagem livre, com um ícone vermelho intermitente e uma mensagem de perigo. É também exibido a distância total percorrida.
- Por fim, é exibido um gráfico relativo à evolução temporal dos batimentos cardíacos. Este gráfico é limitado no eixo Y por 50 e 200 bpm (valores relevantes durante a atividade física). Já no eixo X, exibe os últimos 60 segundos de atividade. É exibida uma mensagem caso não haja dados disponíveis para serem exibidos.

Figura 4.4.5.1: Ecrã de monitorização de atividade



Para que seja possível ilustrar os diferentes campos, é feito uso de algumas bibliotecas. O mapa é desenhado com recurso à biblioteca *FlutterMap*, que obtém os mapas através do OpenStreetMap de forma gratuita. À medida que chegam novas coordenadas, o mapa é centrado automaticamente através da função `mapController.move`.

O botão de pausa está associado a uma variável booleana (*isPaused*), que quando é colocada como *true*, a variável *seconds* deixa de ser atualizada, parando o tempo. Os dados provenientes do broker MQTT são ignorados, com exceção dos relativos à deteção de quedas e obstáculos, que por motivos de segurança, continuam a ser recebidos.

Ao premir o botão concluir, a atividade é encerrada, pedindo ao utilizador para nomear a mesma. Os dados relativos à sessão são enviados ao Node-Red através de HTTP e as *SharedPreferences* são limpas através da função `SessionManager.clear()`. Após este processo, o utilizador é encaminhado para o ecrã do histórico.

Por fim, de forma a prevenir saídas acidentais, caso um utilizador clique no botão de retrocesso com uma atividade a decorrer, é apresentada uma caixa de diálogo a informar o utilizador de que se proceder à saída do ecrã de monitorização o seu progresso não será salvo.

4.4.5.1 Matriz térmica

Devido à complexidade com que tem de ser tratada, a matriz térmica (Fig. 4.4.5.1.1) exige uma explicação detalhada. A aplicação recebe do sensor uma grelha de valores binários com 32x24 temperaturas (768 no total). A sua resolução é baixa, mas suficiente para detetar a presença de pessoas e monitorizar a temperatura corporal. A *app* tem de converter esses valores binários num valor de temperatura e representá-los visualmente.

Para descodificar o *payload* binário, a aplicação faz uso da classe *ByteData*, que permite a leitura de blocos de memória. Oferece como argumento, à função que recolhe os dados, *i*4* (*offset* a multiplicar pelo tamanho de cada bloco) e *Endian.little*, o que garante uma leitura correta dos dados.

Após a descodificação dos dados, são descartados valores fora do intervalo de 15 a 60 graus Celsius, por serem considerados ruído neste contexto. É também calculada a temperatura mínima, máxima e média. Quando é desenhada a matriz térmica, cada pixel com valores de ruído é substituído pelo valor da média.

Para mapear cada valor de temperatura a uma cor, é necessário normalizar cada temperatura para um valor entre 0 e 1. Para isso, é utilizada uma normalização que utiliza o valor mínimo e máximo do *frame* atual. Isso faz com que o pixel mais frio tenha o valor 0 e o mais quente o valor 1. A função *getColor* transforma esses valores numa cor. De seguida, cada pixel é desenhado como um retângulo pelo Flutter, sendo colorido com a cor retornada pela função *getColor*. Para evitar diferenças abruptas entre pixéis, é aplicado um filtro de desfoque. O tamanho de cada pixel é calculado dinamicamente tendo em conta o espaço disponível no ecrã, garantindo que a imagem se adapta a diferentes resoluções de ecrã.

Figura 4.4.5.1.1: Matriz térmica



4.4.6 Configuração via QR Code

Esta funcionalidade da aplicação também merece especial atenção pois é ele que permite que os 3 microcontroladores tenham acesso às credenciais de WiFi. A troca das credenciais é feita via BLE e não mete em causa uma atividade que esteja em curso.

O processo começa com o utilizador a apontar a câmara do telemóvel para o código QR presente na caixa dianteira. A aplicação utiliza a classe MobileScanner para aceder à câmara e decodificar o código QR assim que este entre em campo de visão. O payload do código contém o nome das placas a configurar. Depois de obtidos os nomes, o utilizador é encaminhado para o ecrã de configuração das credenciais de WiFi.

No ecrã de configuração (Fig 4.4.6.1) são apenas pedidos o SSID e a *password* da rede que se pretende utilizar. O IP do *broker* MQTT é inserido diretamente no código da aplicação, garantindo assim que está correto independentemente do utilizador. Após o utilizador clicar no botão “Enviar para as placas”, é invocado o método `enviarDadosBLE()`, que arranca com a comunicação via BLE.

O scan BLE é então iniciado (*timeout* de 90 segundos), sendo a aplicação responsável por verificar se as placas que se pretendem configurar estão entre os dispositivos encontrados. Esta verificação é feita através dos nomes presentes no *payload* do código QR ou através dos UUID's de serviço anunciados por cada microcontrolador.

Quando uma placa é encontrada, é lançado um mecanismo de configuração assíncrono, que não bloqueia o *scan*, nem necessita de esperar que as outras placas sejam encontradas. Este paralelismo é garantido através de dois sets:

- ***processingMacs***: contém os MAC's das placas que ainda estão a ser configuradas.
- ***configuredMacs***: contém os MAC's das placas já configuradas.

A função `configurarPlaca()` executa a configuração individual de uma placa, seguindo a seguinte lógica (máximo de 3 tentativas):

1. Conecta-se através de `device.connect()`.
2. Descobre os serviços e características disponíveis na placa através de `discoverServices()`.
3. Filtra os UUID dos serviços que são conhecidos, sendo os restantes ignorados.
4. É escrito o valor correspondente a cada característica (SSID, *password*, IP do *broker*). Esta escrita é efetuada com um *delay* entre cada característica de forma a permitir que a placa processe um valor antes de receber o próximo.
5. Desconecta-se através de `device.disconnect()`.

Se as 3 tentativas falharem, o MAC da placa correspondente é removido do *processingMacs*, permitindo ao scan que volte a encontrar a placa. A cada placa corretamente configurada, o contador *configuredCount* é incrementado. Quando este contador atinge o valor 3, o *scan* é parado imediatamente e é exibida uma mensagem de confirmação da configuração das placas.

Figura 4.4.6.1: Ecrã de configuração das credenciais de WiFi



4.4.7 Histórico de Atividades

Este ecrã permite ao utilizador consultar atividades anteriores, assim como eliminar as que não têm interesse. Os dados relativos às atividades são obtidos através do Node-Red, via HTTP GET. A resposta do Node-Red é um *array* JSON onde cada elemento corresponde a uma atividade guardada. O *parsing* é feito no método *loadActivities()* que constrói objetos *ActivityData* a partir de cada entrada do *array*. Cada atividade é identificada com um *id*, que corresponde ao seu *timestamp* de criação.

As atividades são então listadas no ecrã (Fig. 4.4.7.1), apresentando o seu nome e o resumo das suas principais métricas. O toque numa das atividades faz com que a aplicação navegue para o ecrã *ActivityDetailScreen*, que apresenta uma atividade em maior detalhe. Um toque mais longo, ativa a seleção múltipla de atividades, que através do método *apagarMultiplas()*, podem ser apagadas em conjunto (é apresentado uma caixa de diálogo a pedir a confirmação da eliminação). As atividades são eliminadas através de um HTTP DELETE (um de cada vez para o caso de eliminação múltipla) enviado ao Node-Red, que envia um objeto JSON com o identificador da atividade a eliminar.

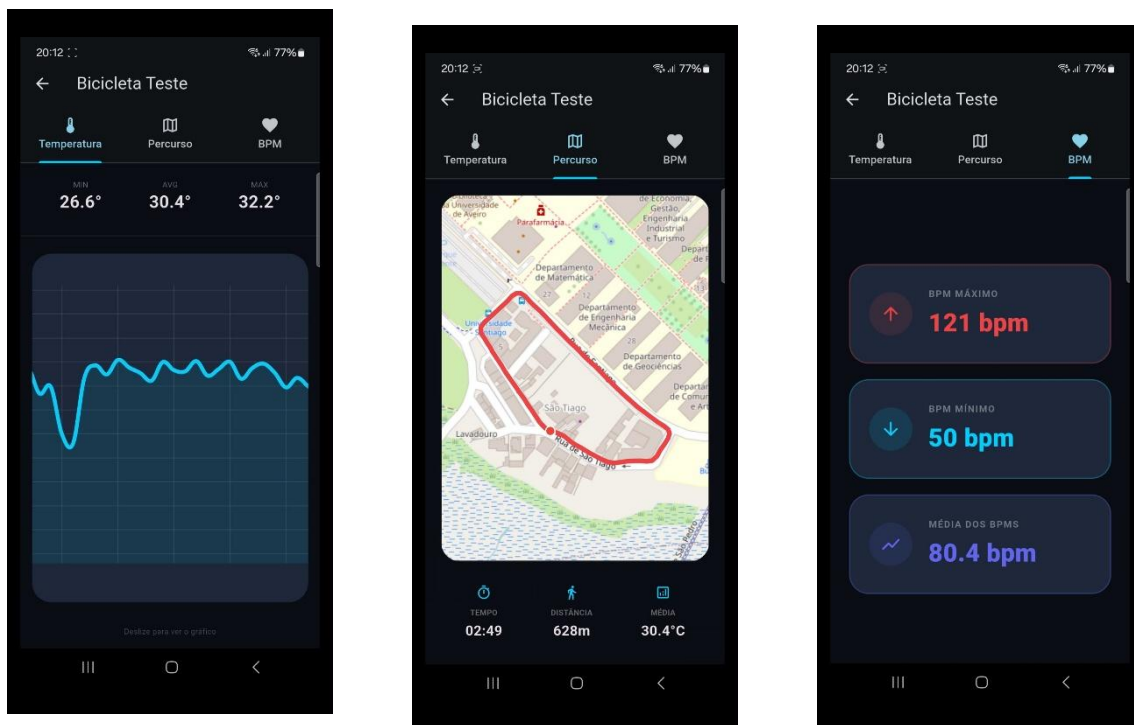
Figura 4.4.7.1: Ecrã de todas as atividades concluídas



O ecrã que exibe uma atividade em detalhe está dividido em 3 separadores (Fig 4.4.7.2). O primeiro exibe estatísticas relativas à temperatura, como a temperatura média, mínima e máxima, assim como um gráfico retratando a evolução da mesma. O segundo separador exibe um mapa do percurso e o terceiro estatísticas relativas aos batimentos cardíacos (semelhante às da temperatura, mas calculadas localmente).

Para o gráfico que demonstra a evolução da temperatura, em vez de comprimir todos os valores no espaço disponível (tornaria o gráfico ilegível para sessões longas), o espaçamento entre pontos é adaptado à duração da atividade, permitindo o *scroll* horizontal. O mapa do percurso é desenhado de forma semelhante ao do ecrã de monitorização (subcapítulo 4.4.5), com a diferença de que o *zoom* não está centrado no ponto atual do utilizador (a atividade já terminou), mas sim numa definição que permita visualizar todo o percurso. São também marcados o início e o fim do percurso e é permitido a navegação do utilizador pelo mapa, de forma a poder explorar diferentes partes do trajeto. Se não existirem dados GPS relativos à atividade, é apresentada uma mensagem informativa no lugar do mapa.

Figura 4.4.7.2: Separadores para visualizar uma atividade já concluída



4.5 Backend

A infraestrutura de *backend* da *Mobile Heart Box* foi inteiramente alojada numa Máquina Virtual no Microsoft Azure e provisionada através de orquestração de contentores com a ferramenta Docker. Dentro desse servidor, temos em execução contínua quatro serviços principais da nossa arquitetura: Eclipse Mosquitto, Node-RED, InfluxDB e Grafana. O funcionamento do nosso backend divide-se em três partes, apresentadas de seguida.

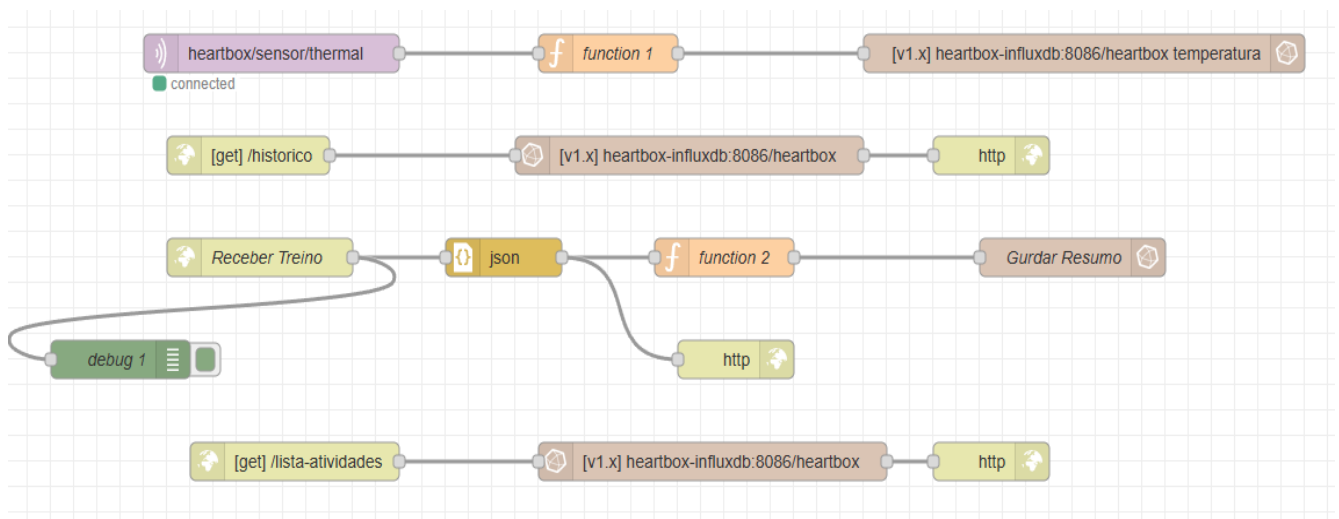
4.5.1 Receção de Dados em Tempo Real (Mosquitto)

Para gerir a telemetria em tempo real, implementou-se o Mosquitto, serviço este que assegura a comunicação de baixa latência baseada no modelo Publish/Subscribe. Esta escolha deve-se ao facto deste serviço fazer uso do protocolo MQTT, escolhido para este projeto devido à sua leveza e eficácia em contextos de IoT com recursos limitados. Em vez de os microcontroladores ESP32 terem de estabelecer uma ligação pesada e direta com o telemóvel do utilizador, eles limitam-se a "publicar" as suas leituras num canal específico (como, por exemplo, o tópico *heartbox/sensor/thermal*). O Mosquitto atua assim como um "controlador de tráfego", recebendo a informação bruta da bicicleta (*Edge Computing*) e reencaminhando-a imediatamente para quem subscreve o tópico.

4.5.2 Middleware do Sistema (Node-RED)

O Node-RED (alojado na porta 1880) atua como o motor lógico e *middleware* central do sistema. Através do fluxo definido, este serviço unifica recepção de dados em tempo real (via MQTT) e o armazenamento dos resumos dos treinos no final da atividade (via API REST). Na seguinte imagem mostramos o desenho do fluxo do Node-RED:

Figura 4.5.2.1: Fluxo do Node-RED



Processamento da Telemetria em Tempo Real

O sistema subscreve ativamente o tópico MQTT *heartbox/sensor/thermal* com Qualidade de Serviço nível 2 (QoS 2). O bloco lógico interno interceta os payloads síncronos recebidos em formato string, converte-os de imediato em valores numéricos e guarda-os diretamente na tabela de medição *temperatura*.

Implementação de API REST para a Gateway Móvel

Foram provisionados nós HTTP para satisfazer os pedidos assíncronos da aplicação Flutter:

- POST /guardar: O endpoint designado "Receber Treino" recolhe a totalidade do ficheiro JSON agregado no momento de conclusão da atividade. O Node-RED recebe este ficheiro e calcula, do lado do servidor, as estatísticas finais (mínimo, máximo e média para os batimentos cardíacos e temperatura). A estrutura final consolida variáveis de contexto (duração, distância e coordenadas da rota) e guarda o registo integral na base de dados *heartbox*.
- GET /lista-atividades: Para alimentar o ecrã de histórico da aplicação, esta rota executa uma pesquisa na base de dados (`SELECT * FROM heartbox ORDER BY time DESC`), devolvendo a lista de eventos ordenada do treino mais recente para o mais antigo.
- GET /historico: Solicita à base de dados os detalhes de uma atividade específica selecionada pelo utilizador.

4.5.3 Persistência de Séries Temporais (InfluxDB)

A persistência dos dados cronológicos é assegurada pelo InfluxDB, uma base de dados otimizada para séries temporais. Operando na porta 8086, esta funciona como o ponto final de persistência exclusivo para o middleware Node-RED. A escolha por uma Base de Dados de Séries Temporais em detrimento de uma base de dados relacional tradicional (como MySQL) deve-se à natureza da telemetria recolhida na Mobile Heart Box. Durante a atividade desportiva, os sensores (como o AD8232 e o MLX90640) geram um volume massivo de medições contínuas, todas associadas a um carimbo de tempo exato (timestamp). O InfluxDB permite indexar e comprimir estes dados temporais com elevada eficiência, garantindo velocidades de escrita elevadas sem bloquear o sistema.

4.5.4 Monitorização (Grafana)

Para efeitos de monitorização, implementou-se o Grafana na porta 3000, cuja execução no contentor está estritamente condicionada à inicialização prévia do InfluxDB. A figura seguinte demonstra o dashboard em funcionamento, ilustrando a marcação contínua da temperatura corporal ao longo do tempo (métrica temperatura.mean), o que comprova a eficácia da gravação das séries temporais:

Figura 4.5.4.1: Leitura de valores de temperatura num Dashboard Grafana



4.6 Desafios e Soluções

Neste subcapítulo pretendemos demonstrar alguns dos desafios que foram surgindo ao longo do desenvolvimento da Mobile Heart Box e de que forma foram ultrapassados.

O principal desafio a surgir esteve relacionado com a parte central do nosso projeto, a medição de batimentos cardíacos. Inicialmente, e como será referido no capítulo 5, o sensor utilizado para efetuar estas medições era um sensor PPG. No entanto, ficou imediatamente claro que esse sensor não seria uma solução viável, uma vez que era necessário o dedo do utilizador estar sempre em contacto com o sensor, e além disso, o sensor tinha imensas dificuldades em fazer recolhas de dados fidedignas em mobilidade, bastando pequenas vibrações para afetar os dados recolhidos. De seguida, testou-se um sensor de batimentos cardíacos sem contacto. Este era, para nós, a solução que fazia mais sentido para um contexto que exige fazer medições em movimento. No entanto, o sensor ao qual tivemos acesso, apenas conseguia efetuar medições até aos 100bpm, o que é facilmente ultrapassado quando se está a praticar desporto. Sendo assim, a solução encontrada foi utilizar o sensor AD8232, que determina os batimentos cardíacos através de impulsos elétricos. Mesmo após a escolha deste sensor como componente da solução final, surgiu a necessidade de integrar os elétrodos de uma forma a que fosse possível recolher os dados pretendidos. A solução encontrada foi a descrita no subcapítulo 4.2.3, posicionando os elétrodos numa posição estratégica e fazendo uso de fios de cobre para recolher os impulsos elétricos.

5 Testes e Resultados

A fase de testes foi conduzida de forma progressiva, iniciando com a validação individual de cada componente e evoluindo para testes de integração do sistema completo. A metodologia adotada passou por duas fases distintas: numa primeira fase, os dados eram validados diretamente através de subscrições aos tópicos do broker MQTT, verificando se os sensores transmitiam corretamente as leituras; numa segunda fase, os testes passaram a ser realizados com recurso à aplicação móvel desenvolvida, que permitia visualizar os valores em tempo real de forma mais intuitiva e próxima da experiência real do utilizador.

5.1 Testes Funcionais

5.1.1 Sensor de Batimentos Cardíacos (AD8232)

A integração do sensor AD8232 com os eléktodos posicionados no guiador, através de fio de cobre enrolado, revelou-se uma solução funcional, mas com algumas limitações práticas. O fio de cobre, sendo um excelente condutor de impulsos elétricos, permitiu na maioria das situações a recolha de valores de BPM credíveis e dentro dos intervalos esperados para a atividade física praticada. No entanto, verificou-se que, pontualmente, os valores transmitidos não eram os mais ajustados à realidade, situação que se ficou a dever à natureza da ligação entre o utilizador e os eléktodos — dependente do contacto físico com o guiador — que nem sempre era consistente ao longo do percurso. Ainda assim, no geral, o sensor cumpriu o seu propósito, sendo os valores incorretos facilmente identificáveis pelo utilizador na interface da aplicação.

5.1.2 Câmara Térmica (MLX90640)

Os testes com a câmara térmica MLX90640 exigiram um processo de calibração física da posição da caixa dianteira. Para que as leituras de temperatura fossem credíveis, foi necessário ajustar repetidamente a orientação da caixa de forma a garantir que a câmara captava uma zona corporal próxima da face do ciclista, onde a temperatura à superfície é mais representativa da temperatura real do utilizador. Após este ajuste, os valores obtidos revelaram-se consistentes. Pontualmente, devido a oscilações do percurso ou a outros fatores externos, eram registados valores atípicos.

No entanto, estes valores eram mitigados no histórico da atividade através do cálculo da temperatura média, tornando o impacto desses picos residual na análise final.

5.1.3 Sensor IMU

O sensor IMU foi o componente que apresentou os resultados mais consistentes ao longo de todos os testes realizados. Foram simulados vários cenários de queda, com diferentes ângulos e intensidades, e em todos os casos o sensor reagiu corretamente, disparando o alerta correspondente na aplicação móvel. Não foram registados falsos positivos durante os percursos de teste, o que demonstra que o limiar de deteção de 60° de inclinação lateral definido no firmware se revelou adequado ao contexto de utilização.

5.1.4 Sensor de Proximidade (mmWave HMMD)

Os testes ao sensor de proximidade revelaram algumas limitações, tanto de natureza técnica como de implementação física. Em termos de comportamento, verificou-se uma assimetria na velocidade de resposta do sensor: quando no estado livre, a transição para o estado de perigo aquando da deteção de um obstáculo era rápida e eficaz; no entanto, a transição inversa — de perigo para livre após a saída do obstáculo — revelou-se mais lenta, o que por vezes causava alguma confusão ao utilizador.

Foi ainda identificado um erro de implementação física da nossa parte: o orifício destinado ao sensor na caixa traseira foi posicionado demasiado para baixo, o que fazia com que, em determinadas condições, o pneu traseiro da própria bicicleta ou a superfície da estrada fossem detetados como obstáculos, gerando falsos alertas de perigo. Esta limitação foi identificada durante os testes e constitui uma melhoria prioritária para versões futuras do hardware, conforme referido no capítulo 6.1.

5.1.5 Módulo GPS

O módulo GPS foi o componente que apresentou o comportamento mais previsível e estável ao longo de todos os testes. A deteção de coordenadas e o traçado do percurso na aplicação móvel funcionaram sempre de forma correta e precisa. Foi, no entanto, identificada uma limitação expectável deste tipo de tecnologia: o módulo não conseguia adquirir sinal em ambientes interiores ou em zonas muito fechadas, sendo necessário que o utilizador se encontrasse no exterior para que o GPS ficasse operacional. Esta limitação não compromete o caso de uso principal do sistema, uma vez que a Mobile Heart Box se destina precisamente à utilização em contexto exterior.

5.2 Testes de Integração e Comunicação

5.2.1 Latência MQTT

Durante os testes de integração, foram detetados pontualmente alguns atrasos na transmissão de dados via MQTT entre os microcontroladores e a aplicação móvel. Estes atrasos, embora não frequentes, foram atribuídos a instabilidades na rede WiFi utilizada durante os percursos de teste, e não a limitações do próprio protocolo ou da implementação. Em condições de rede estável, a latência observada foi reduzida e não comprometeu a experiência do utilizador nem a utilidade das leituras em tempo real.

5.2.2 Estabilidade WiFi

A ligação WiFi entre os microcontroladores e o broker MQTT manteve-se estável ao longo de todos os percursos de teste realizados. Não foram registadas desconexões inesperadas que afetassem o funcionamento do sistema durante a atividade, o que valida a robustez do mecanismo de reconexão automática implementado no firmware e na aplicação.

5.2.3 Configuração via QR Code e BLE

O processo de configuração das credenciais WiFi via QR Code e BLE funcionou de forma satisfatória na generalidade dos testes. Na maioria das situações, as três placas eram configuradas em simultâneo e de forma rápida, sem necessidade de intervenção adicional por parte do utilizador. Nos casos em que alguma das placas não estabelecia ligação de imediato, o sistema solicitava um novo scan do QR Code, conseguindo concluir a configuração ao fim de alguns segundos adicionais. Este comportamento, embora menos fluido, não comprometeu a usabilidade do sistema.

5.3 Testes Não Funcionais

5.3.1 Autonomia Energética

Apesar de não terem sido realizados percursos com a duração máxima estimada de 12 horas, os testes efetuados permitiram concluir que o consumo energético do sistema é bastante moderado. Em atividades com várias horas de duração, o nível de bateria das powerbanks manteve-se confortavelmente adequado, sem comprometer o funcionamento do sistema. Estes resultados são consistentes com as estimativas apresentadas no subcapítulo 4.2.5 e transmitem confiança na autonomia do sistema para os casos de uso típicos.

5.3.2 Usabilidade

Foram realizados testes de usabilidade com utilizadores externos ao grupo de desenvolvimento. O feedback recolhido foi globalmente positivo: os utilizadores conseguiram, de forma autónoma, iniciar uma atividade e interagir com as funcionalidades principais da aplicação. Nos casos em que surgiram dúvidas ou dificuldades pontuais, o manual de utilizador disponível na aplicação revelou-se um recurso eficaz para as esclarecer, cumprindo o objetivo de tornar o sistema acessível a utilizadores sem conhecimentos técnicos aprofundados.

5.3.3 Desempenho do Backend

A infraestrutura de backend, composta pelo Mosquitto, Node-RED, InfluxDB e Grafana, funcionou de forma estável e sem falhas ao longo de todos os testes realizados. Todos estes serviços correram em contentores Docker alojados numa Máquina Virtual da Microsoft Azure, que se manteve disponível e acessível durante todo o período de testes, sem interrupções de serviço registadas. O fluxo de dados entre a aplicação móvel, o broker MQTT e a base de dados foi sempre processado corretamente, e a consulta do histórico de atividades na aplicação devolveu sempre os dados esperados de forma consistente.

6 Conclusões

O projeto Mobile Heart Box demonstrou com sucesso a viabilidade de um ecossistema integrado de IoT para a segurança e monitorização desportiva avançada. Numa área de mercado tipicamente marcada pela fragmentação de dispositivos e pela dependência de plataformas comerciais fechadas, este trabalho provou que é possível construir uma arquitetura 100% autónoma, garantindo a soberania e a privacidade dos dados do atleta.

Ao nível de hardware, o facto de termos decidido distribuir o processamento por três microcontroladores ESP32 revelou ser uma decisão arquitetural acertada. Tudo isto, permitiu gerir, sem bloqueios de processamento, dados de naturezas muito distintas, e com muito sucesso.

A infraestrutura de *backend* provou ser altamente robusta no tratamento do fluxo de dados. A implementação do protocolo MQTT (via Mosquitto) garantiu a baixa latência necessária para renderizar a telemetria e os alertas de proximidade em tempo real na aplicação móvel. Em paralelo, a utilização do Node-RED assegurou a entrega e persistência fiável dos resumos de treino no InfluxDB, otimizando o armazenamento de séries temporais.

Em suma, em vez de se focar unicamente na procura de uma precisão absoluta em leituras isoladas, o sistema mostrou-se extremamente eficaz na deteção de tendências contínuas. A capacidade de cruzar a alternância dos dados do ritmo cardíaco, o mapa térmico corporal e os alertas ambientais do radar possibilitaram criar uma rede de segurança ativa real, cumprindo integralmente os objetivos propostos para esta prova de conceito.

6.1 Trabalho Futuro

Apesar de o protótipo funcional ter validado a arquitetura planeada, o projeto abre diversas vias de otimização e expansão para aproximar o sistema de um produto final (*market-ready*):

- **Ergonomia e Integração Física:** Diminuir o tamanho da caixa dianteira de forma a ocupar menos espaço no guiador, assim como torná-la mais resistente a condições adversas. Para além disso, substituir os fios de cobre por uma solução mais cómoda e eficiente. Relativamente ao sensor de proximidade Wave, o orifício destinado à sua deteção deveria ter sido colocado mais acima na caixa, de forma a evitar leituras incorretas de obstáculos, como a estrada ou o pneu da bicicleta.
- **Resiliência de Rede:** Atualmente, o sistema requer conectividade contínua para o envio de dados. Em percursos de montanha (onde a cobertura de rede móvel é escassa), planeia-se a implementação de módulos de memória local (cartão SD). O sistema deverá guardar os dados temporariamente no nó local e forçar a sincronização com a base de dados na Cloud assim que a ligação for restabelecida.
- **Feedback Físico Sensorial:** Expansão do sistema de alertas gerados pelo radar mmWave e pelo detetor de quedas. Para não depender exclusivamente do ecrã do smartphone, o desenvolvimento futuro contempla a integração de motores de vibração no guiador ou de emissores sonoros, garantindo que o ciclista é notificado de perigos sem ter de desviar os olhos da estrada.

Referências

1. Church, T. S., Thomas, D. M., Tudor-Locke, C., Katzmarzyk, P. T., Earnest, C. P., Rodarte, R. Q., Martin, C. K., Blair, S. N., & Bouchard, C. (2011). *Trends over 5 decades in U.S. occupation-related physical activity and their associations with obesity*. PLoS ONE, 6(5), e19657.
2. Moreno-Llamas, A., García-Mayor, J., & De la Cruz-Sánchez, E. (2020). *The impact of digital technology development on sitting time across Europe*. Technology in Society, Volume 63, 101406.
3. Romero-Ortuno, R., Cogan, L., Cunningham, C. U., & Kenny, R. A. (2011). Time trends in leisure-time physical activity and physical fitness in elderly people: 20-year follow-up of the Spanish population national health survey. BMC Public Health, 11, 799.
4. Organisation for Economic Co-operation and Development (2019). *Current and Past Trends in Physical Activity in Four OECD Countries*. OECD Health Working Papers.
5. Kirk, M. A., & Rhodes, R. E. (2011). *Occupational Sitting and Leisure-Time Physical Activity: A Systematic Review*. American Journal of Preventive Medicine, 40(3), 306–315.
6. Brownson, R. C., Boehmer, T. K., & Luke, D. A. (2005). *Declining Rates of Physical Activity in the United States: What Are the Contributors?* Annual Review of Public Health, 26, 421–443.
7. Outdoor Foundation, & Outdoor Industry Association. (2024). *Outdoor participation trends report 2024*. Boulder, CO: Outdoor Industry Association.
8. European Outdoor Group, & It's Great Out There Coalition. (2020). *Consumer research on participation in outdoor activities in Europe*. Brussels: European Outdoor Group.
9. Caroline Ribeiro, João Rodrigues, Júlia Abrantes, Mateus Fonte, Nicole Rakov, Theo Paschôa – Grupo 13 da UC de PEGI (2024/2025). *The Mobile Heart Box*. Aveiro: Universidade de Aveiro.
10. https://hexoskin.com/?srsId=AfmBOoo0dWXvQ0lxBP_i4k7AmB2kWKvRHcVZ9wZ3k3N4WIya9B5aqD92